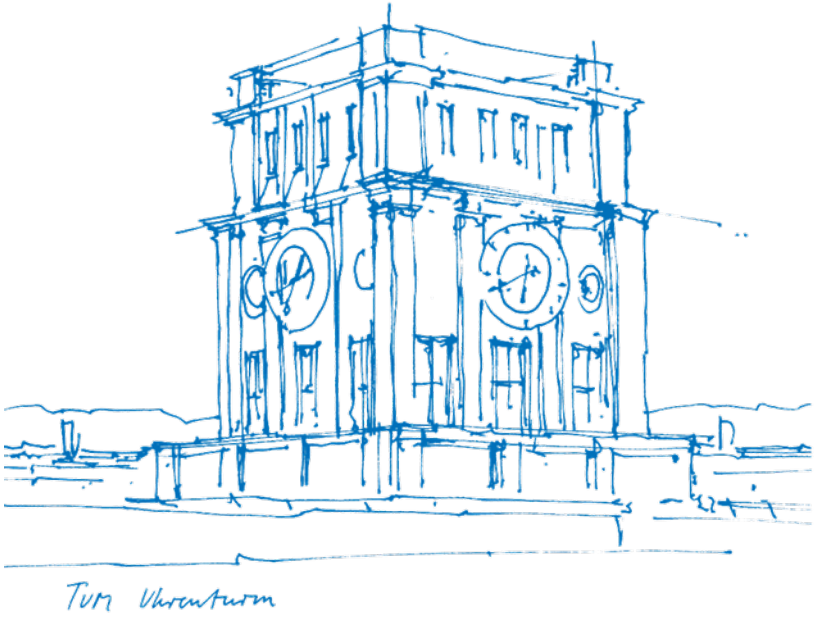
Repetitorium - WS25/26

Patrick Bokelmann, Maxi Heller, Justin Lanfermann, Alisa Parashchenko, Jaden Rotter, Nathan Schmidt, Leon Stark, Isabella Tefera

Grundlagen: Betriebssysteme und Systemsoftware (IN0009/IN0034)

Lehrstuhl für Connected Mobility
School of Computation, Information and Technology
Technische Universität München

Garching, 31.01.2026



Disclaimer:

Bei allen Folien handelt es sich um privat erstellte und verwaltete Folien für GBS im WS25/26.

Sie erheben keinerlei Anspruch auf Vollständigkeit und Korrektheit.

Im Zweifel haben IMMER die offiziellen Vorlesungsmaterialien recht!

© F. Sauter & C. Menges & S. Kappes & J. Schönberger & D. Rentz & A. Stephan & P. Bokelmann & T. Jungmann & N. Schmidt & J. Rotter & A. Scheitler & N. Haunschild & M. Heller & J. Lanfermann & A. Parashchenko & L. Stark & I. Tefera

Speicherverwaltung

http://paging.tum.sexy/

Ziele der Speicherverwaltung

- Verwaltung des physischen Speichers: hier Fokus auf Hautpspeicherverwaltung
- Freispeicherverwaltung
  - Datenstrukturen zur Verwaltung von belegten und freien Speicherbereichen
  - Strategien zur Zuteilung von freien Speicherbereichen
- Virtuelle Adressierung
  - Abbildung von mehreren virtuellen Prozess-Adresseräumen auf physische Speicheradressen
  - insb. Paging:
    - Adressübersetzung, Page-Tabellen
    - Seitenersetzungsstrategien

Anforderungen

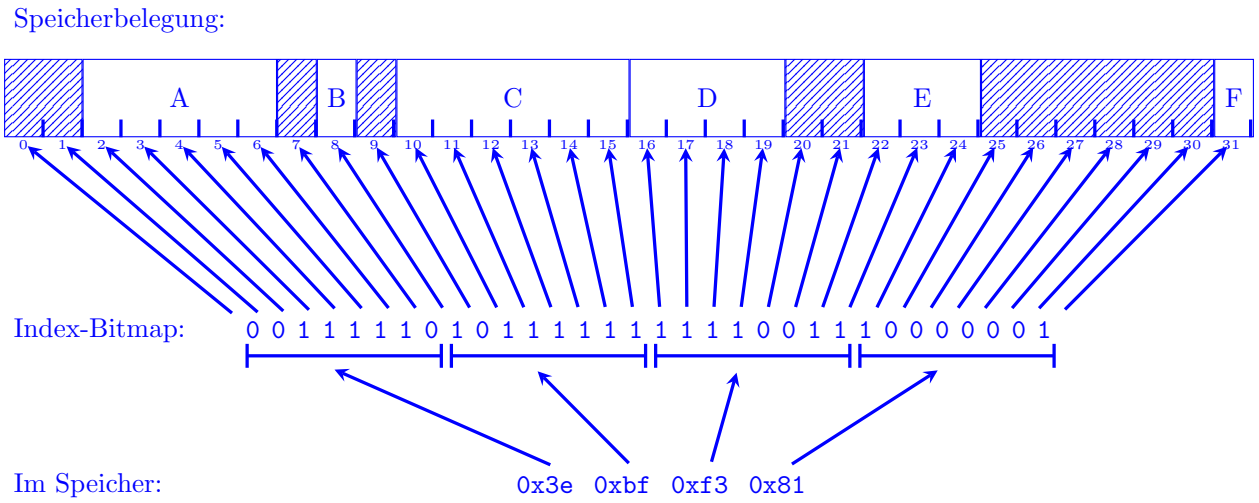
- Ideale Welt: Hauptspeicher (RAM, Main Memory) ist unbeschränkt *groß, schnell, preiswert* und *nicht flüchtig* bei Verlust der Stromversorgung
- Realität: „schnell“zugreifbare Hardware deutlich teurer, als langsame Hintergrundspeicher
- Konsequenzen:
  - Große Datenvolumen werden auf langsamen Hintergrundspeichern gespeichert
  - Zur Verarbeitung müssen die Daten jedoch im Hauptspeicher liegen
  - Hauptspeicher ist eine knappe Ressource, die das Betriebssystem zu verwalten hat

Freispeicherverwaltung

Wie merken wir uns, wo Speicher frei/belegt ist? - Bitmaps

Bitmaps

- Problem: Wahl der Blockgröße (zu klein → große Bitmap, zu groß → Verschnitt)
- Vorteil: Einfacher, schneller Zugriff auf Speicherblöcke fixer Größe
- Nachteil: Lineare Suche für zusammenhängende Blöcke

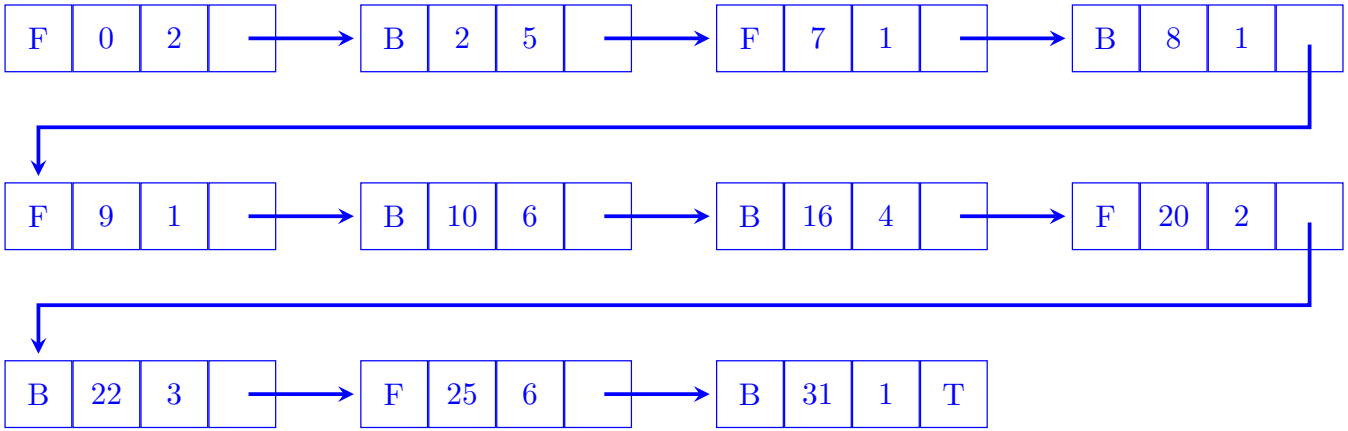


Wie merken wir uns, wo Speicher frei/belegt ist? - Verkettete Listen

Verkettete Listen

- Vorteile: flexible Speicheraufteilung
- Nachteile: Lineare Suche, keine feste Verwaltungsstruktur
- Optimierungen: Getrennte Listen (belegt/frei), Sortierung, Baumstruktur

Einfach verkettete Liste der Speichereinträge:



Eintrag: Status (F/B) | Startindex | Länge | Next-Zeiger

Wie vergeben wir die freien Speicherbereiche?

First-Fit

- Durchsuchen der Liste der Freibereiche, beginnend am Anfang der Liste
  - Der erste Freibereich ausreichender Größe wird belegt
  - Der nicht benötigte freie Speicher wird als neuer, verkleinerter Freibereich der Freibereichsliste zugefügt.
- Vorteil: Sehr einfach, findet schnell einen Freibereich
- Mögliche Probleme?

Next-Fit

Best-Fit, Worst-Fit

Buddy-Algorithmus

Wie vergeben wir die freien Speicherbereiche?

First-Fit

Next-Fit

Variante von First-Fit:

- Durchsuchen der Liste der Freibereiche, beginnend beim als letztes belegten Bereich
- Der erste passende Freibereich wird belegt
- Nicht genutzter Speicher wird als Freibereich der Liste hinzugefügt

Best-Fit, Worst-Fit

Buddy-Algorithmus

Wie vergeben wir die freien Speicherbereiche?

First-Fit

Next-Fit

Best-Fit, Worst-Fit

- Best-Fit
  - Vollständiges Durchsuchen der Liste der Freibereiche
  - Freibereich mit dem geringsten Verschnitt wird belegt.
  - Vor- und Nachteile?
- Worst-Fit
  - Vollständiges Durchsuchen der Liste der Freibereiche
  - Freibereich mit dem größten Verschnitt wird belegt
  - Vor- und Nachteile?

Buddy-Algorithmus

Wie vergeben wir die freien Speicherbereiche?

First-Fit

Next-Fit

Best-Fit, Worst-Fit

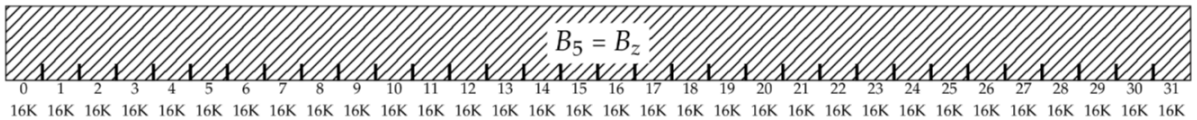
Buddy-Algorithmus

- Suche nach k freien zusammenhängenden Speicherblöcken und der Verwaltung von Speicherbereichen dynamischer Größe
- aufwendige Zusammenführung freier Speicherbereiche
- sucht den kleinstmöglichen Block b, bei dem der angeforderte Speicherbereich $s = 2^k$ kleiner oder gleich $|b|$ ist.

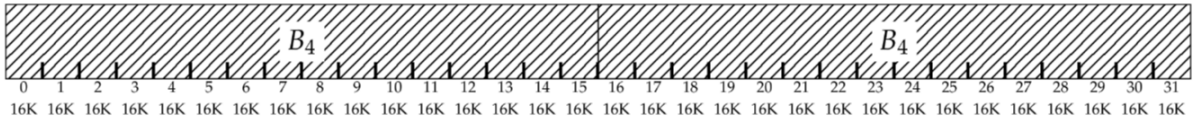
Beispiel Buddy-Algorithmus

Beispiel: `W = alloc(57K); X = alloc(65K); Y = alloc(120K); Z = alloc(99K); free(X); free(Y); free(W); free(Z);` (Blockgröße 16K)

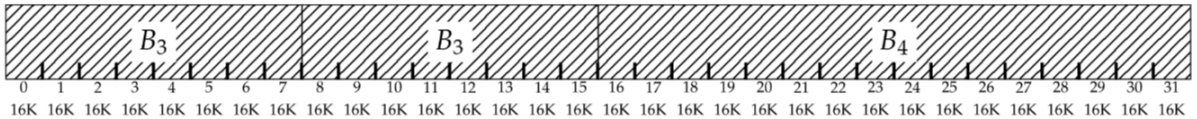
1. Initialzustand



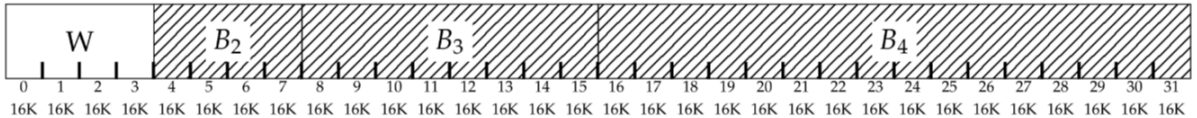
2. Teile ersten Block



3. Teile ersten Block

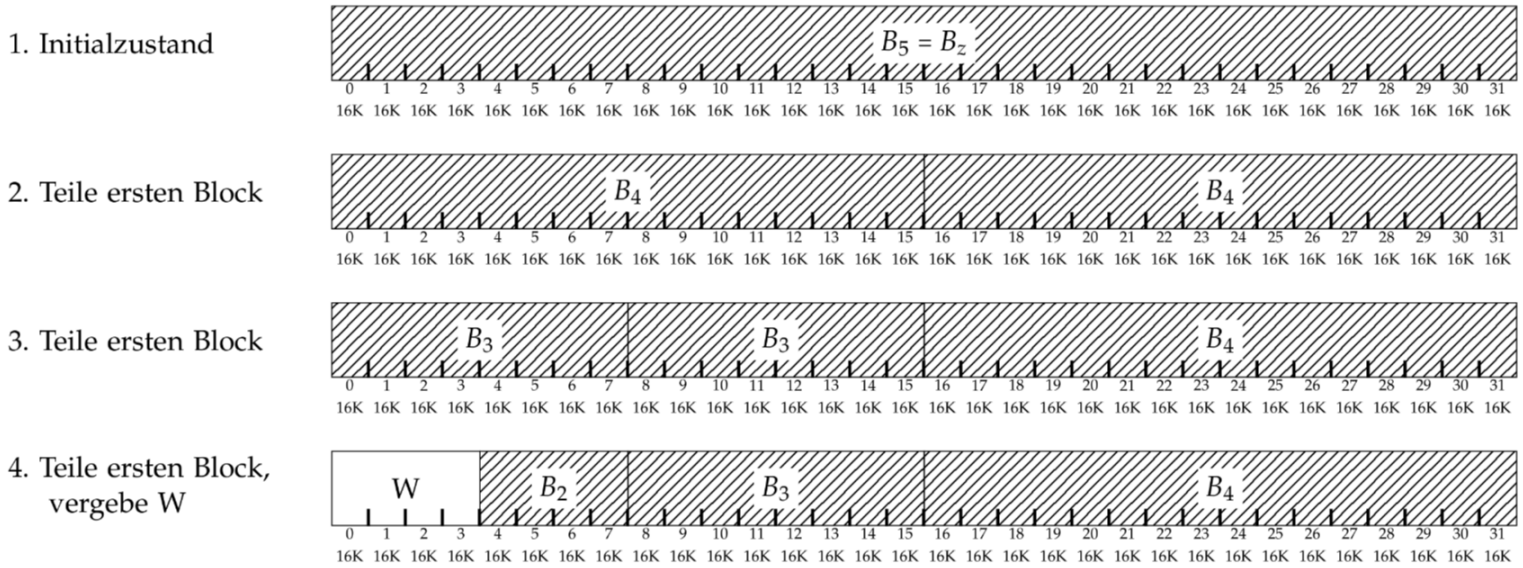


4. Teile ersten Block, vergebe W



Beispiel Buddy-Algorithmus

Beispiel: `W = alloc(57K); X = alloc(65K); Y = alloc(120K); Z = alloc(99K); free(X); free(Y); free(W); free(Z);` (Blockgröße 16K)



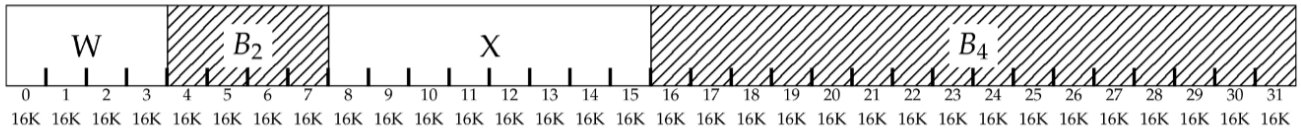
Klassische Fehler:

- nicht Best-Fit zu benutzen
- nicht den am Besten passenden Block ganz links zu nehmen

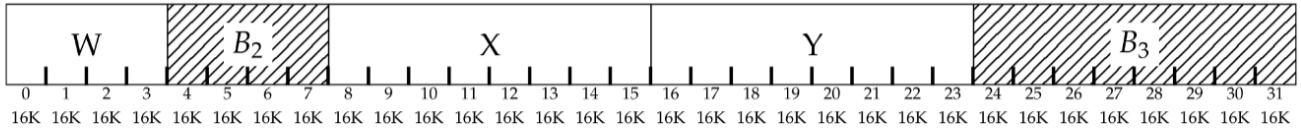
Beispiel Buddy-Algorithmus

Beispiel: `W = alloc(57K); X = alloc(65K); Y = alloc(120K); Z = alloc(99K); free(X); free(Y); free(W); free(Z);` (Blockgröße 16K)

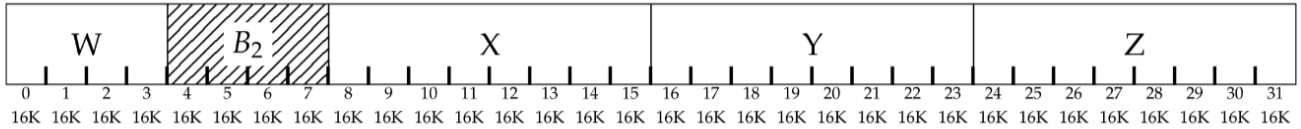
5. Vergebe X



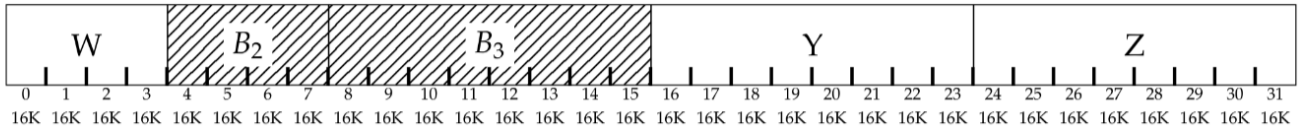
6. Vergebe Y



7. Vergebe Z

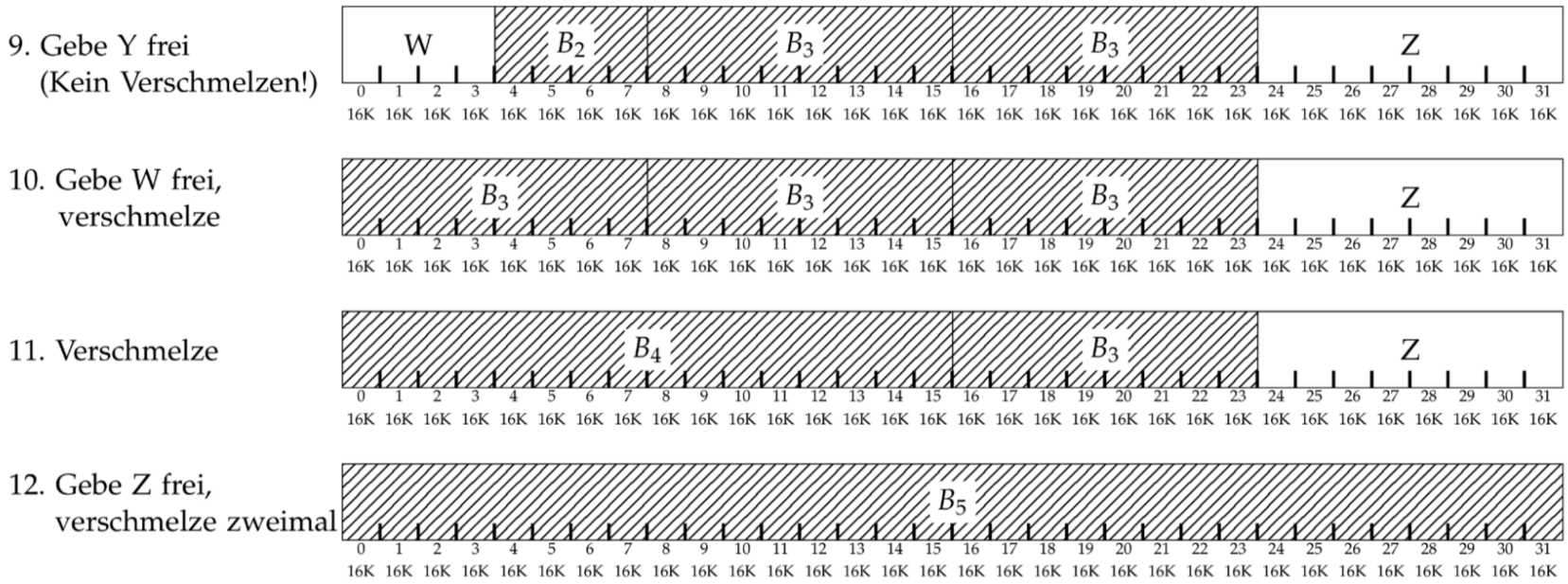


8. Gebe X frei



Beispiel Buddy-Algorithmus

Beispiel: `W = alloc(57K); X = alloc(65K); Y = alloc(120K); Z = alloc(99K); free(X); free(Y); free(W); free(Z);` (Blockgröße 16K)



Klassische Fehler:

- Bereiche zu verschmelzen, die keine Buddies sind
- Unterschiedlich große Bereiche zu verschmelzen

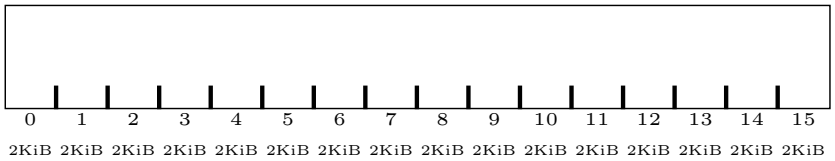
Aufgabe Buddy-Algorithmus:

Gegeben sei der Buddy-Allocator-Algorithmus gemäß Vorlesung, sowie der Pseudocode des folgenden Programms. Gehen Sie von einer Blockgröße von 2048 Bytes (2 KiB) und einer Gesamtspeichergröße von 32768 Bytes (32 KiB) aus.

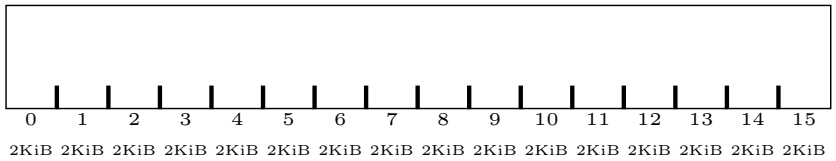
```
A = allocate(4000);
B = allocate(2049);
C = allocate(4096);
free(B);
D = allocate(4242);
free(C);
```

Lösungsweg - Buddy-Algorithmus:

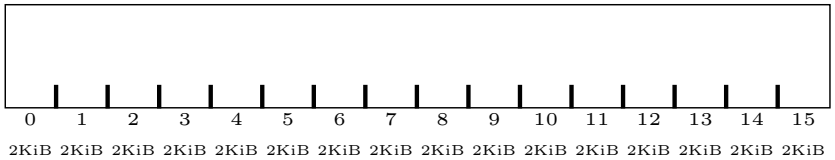
1. Nach `A = allocate(4000);`



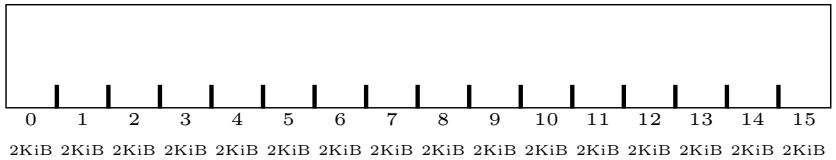
2. Nach `B = allocate(2049);`



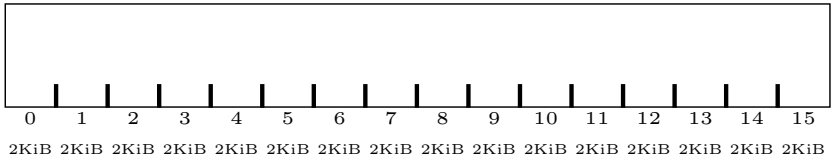
3. Nach `C = allocate(4096);`



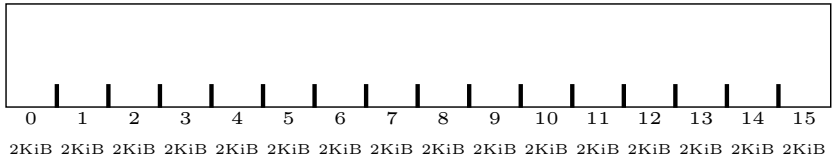
4. Nach `free(B);`



5. Nach `D = allocate(4242);`

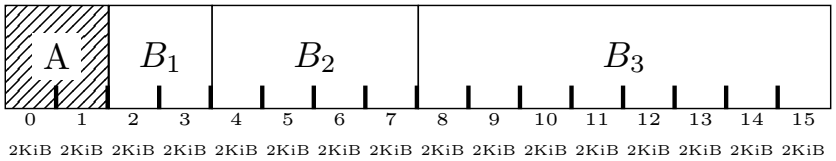


6. Nach `free(C);`

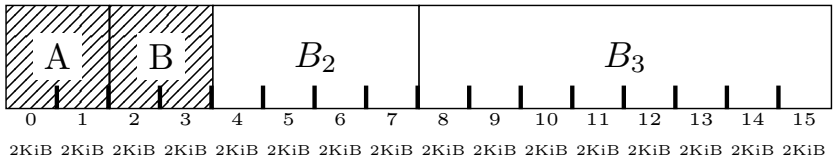


Lösung - Buddy-Algorithmus:

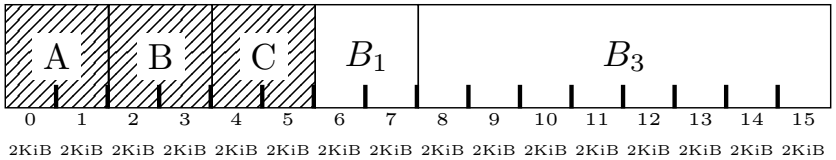
1. Nach `A = allocate(4000);` ⇒ 4096 KiB



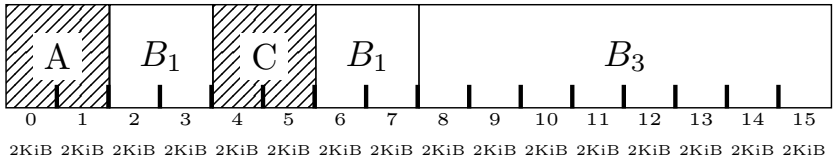
2. Nach `B = allocate(2049);` ⇒ 4096 KiB



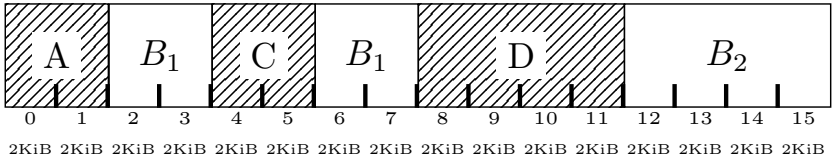
3. Nach `C = allocate(4096);` ⇒ 4096 KiB



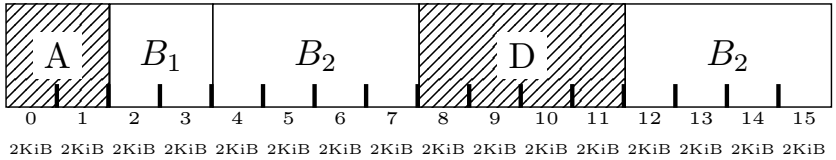
4. Nach `free(B);`



5. Nach `D = allocate(4242);` ⇒ 8192 KiB



6. Nach `free(C);`



Noch mehr Aufgaben? ⇒ Buddy-Algorithmus-Hausaufgabe

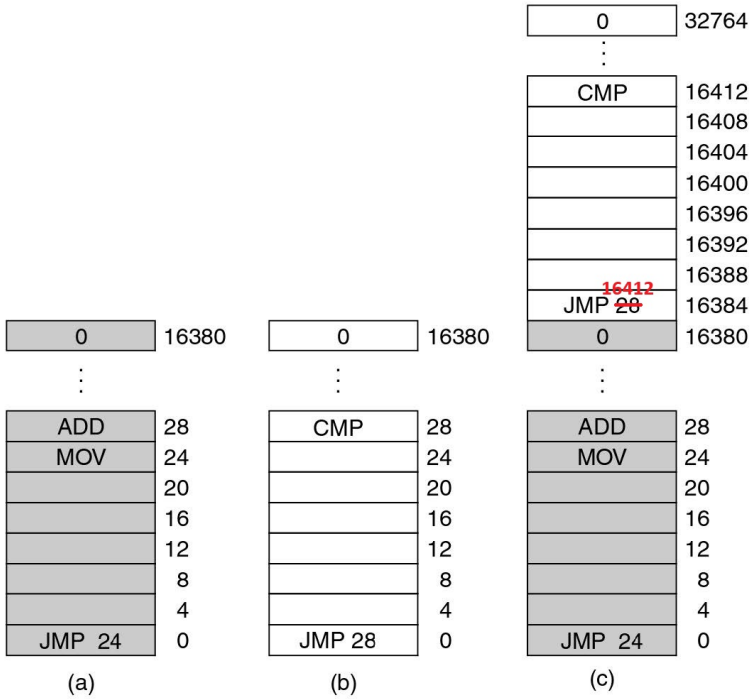
Virtuelle Adressierung

Verwaltung von Adressräumen durch BS

- Physischer Adressraum des Hautspeichers
  - Folge von fortlaufend nummerierten Bytes, beginnend bei 0
  - Die Position eines Bytes bezeichnet seine physische Speicheradresse
- Logischer Adressraum eines Prozesses
  - Menge von adressierbarem Speicher
  - Die Position eines Bytes im logischen Adressraum bezeichnet seine logische Adresse (bzw. virtuelle Adresse)
- Speicherabbildung (Memory Address Translation)
  - Abbildung von logischen Adressen auf physische Adressen

Direkte Adressierung

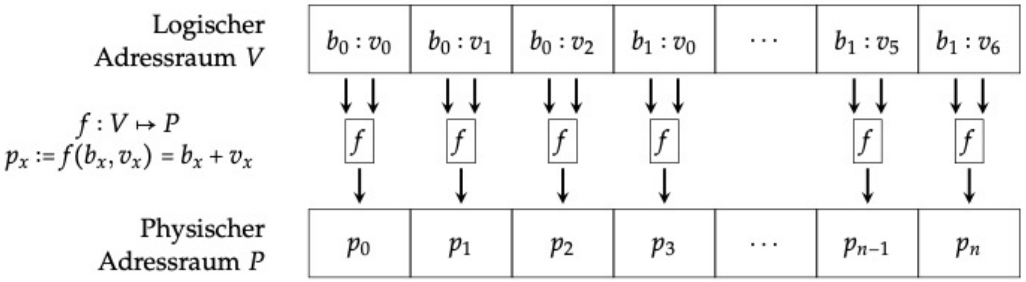
- Phys. Adresse = virt. Adresse: $p_x = v_x$
- Mehrprogrammausführung problematisch:
  - **Swapping**: nur ein Programm im HS $\Rightarrow$ langsam
  - **Relokation**: Adressen beim Laden umschreiben $\Rightarrow$ aufwendig
- Bsp: Programm B wird bei 16380 geladen, aber `JMP 28` zeigt noch auf Adresse 28 $\Rightarrow$ muss zu `JMP 16412` korrigiert werden



(a) Prog A, (b) Prog B, (c) beide geladen

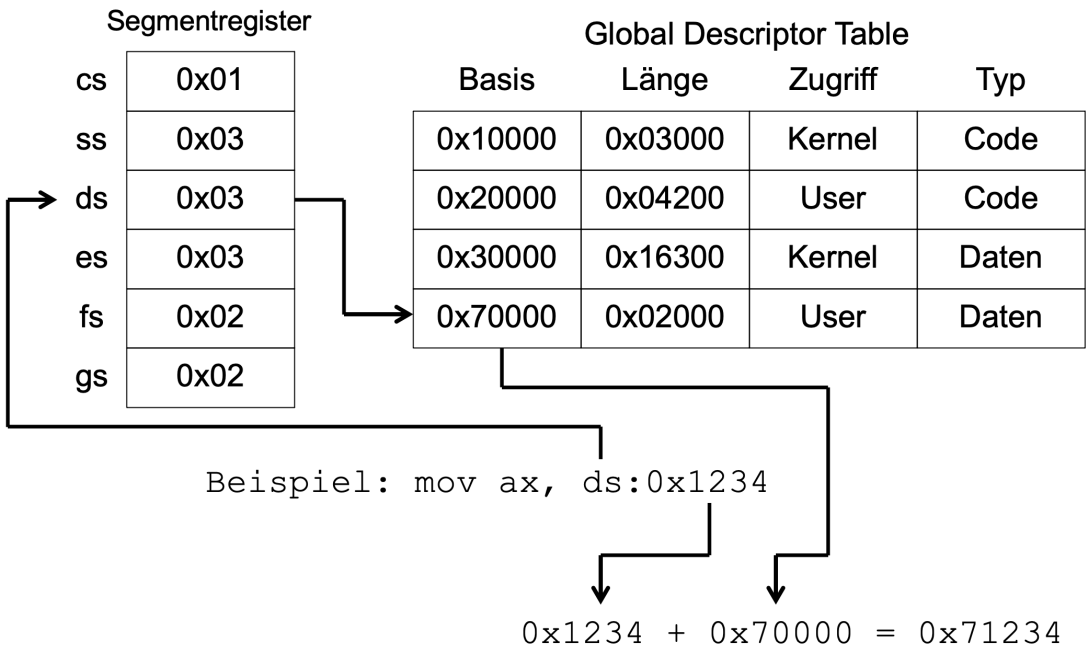
Basisadressierung

- Logischer Adressraum kriegt **Basisadresse** b_x, bei Prozessstart Laden in Basisregister
- Adressberechnung: $p_x := f(b_x, v_x) = b_x + v_x$
- Mehrere Programme ohne Gefahr gleichzeitig im HS, BS kann Programme verschieben
- Nachteil: Addition bei jedem Speicherzugriff



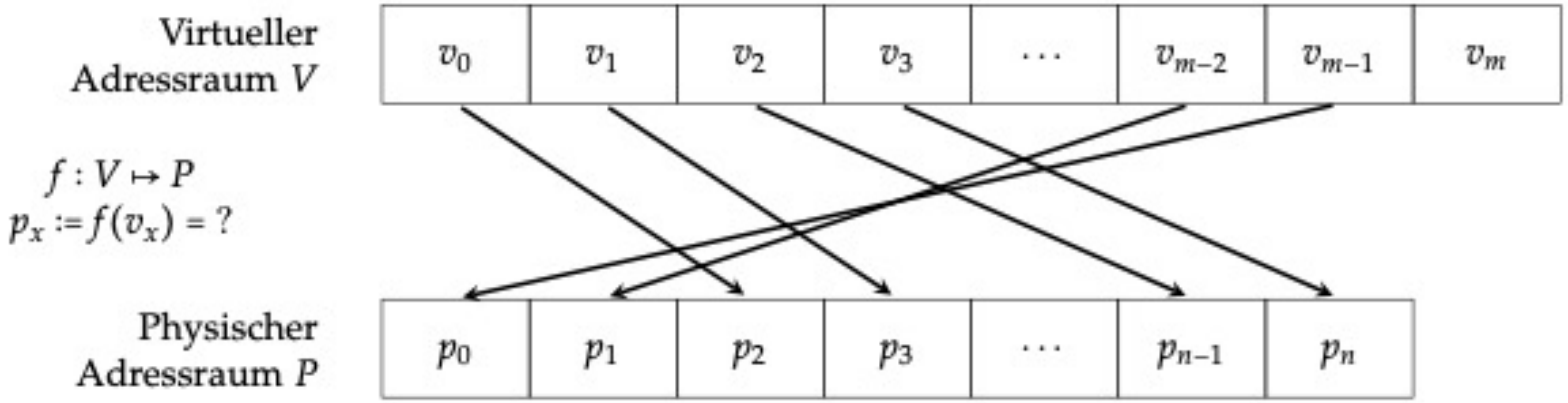
Segmentadressierung

- Unterteilung Adressraum in logische Segmente (Stack, Daten, Code)
- Unterschiedliche Zugriffsrechte pro Segment möglich
- x86: **Segmentregister** $\rightarrow$ Index in **Global Descriptor Table** (GDT)
- GDT-Eintrag: Basis, Länge, Rechte, Typ
- Adresse: `segment:offset`, phys. Adresse = Basis + offset
- Falls offset $>$ Segmentlänge $\Rightarrow$ **Segmentation Fault**



Paging

- Virtuelle Adressräume werden in Seiten aufgeteilt
- Hauptspeicher wird in physische Kacheln aufgeteilt
- **Kachelgröße = Seitengröße**!
- Aufgabe des BS: Abbilden von Seiten auf Kacheln



Paging

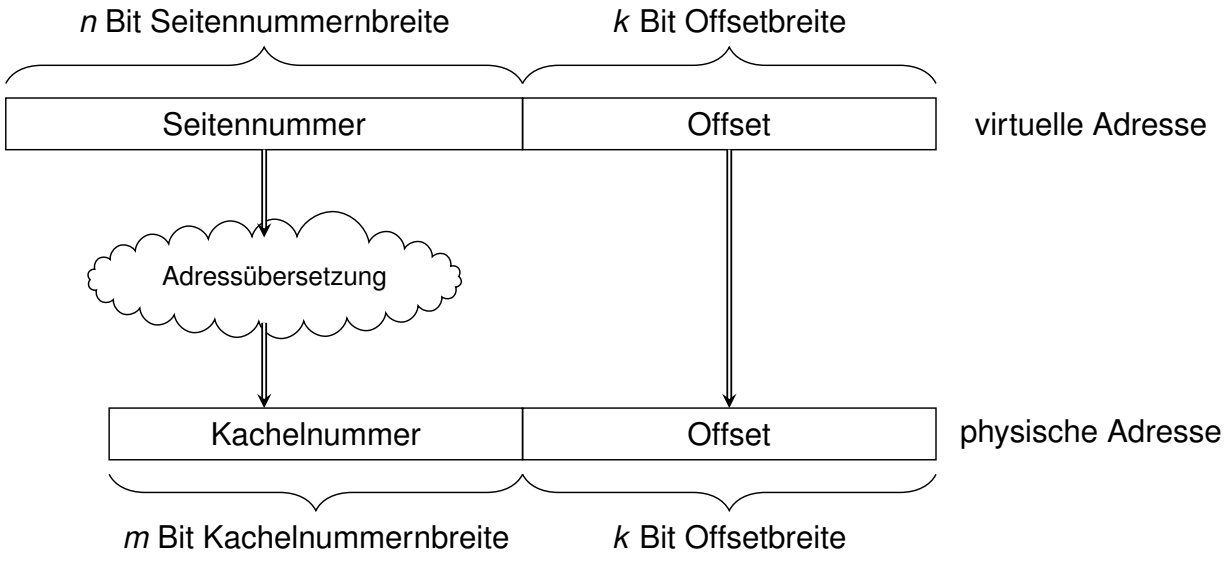


Abbildung: Symbolbild Adressübersetzung.

- Seitennummer: Index in der Page Table
- Page-Table-Eintrag:
  - 1 Eintrag pro Seite
  - Ein Eintrag umfasst i.d.R. 32 Bit bzw. 64 Bit
  - Enthält die Frame-Nummer für die Seite (falls vorhanden)
- Adressabbildung: Seitennummer wird durch die Kachelnummer ersetzt

Paging - Page Table Einträge

$\exists$ **Seitentabelle** für jeden Prozess: Verwaltet Zuordnung Seite $\rightarrow$ Kachel

- Bei Prozesswechsel: Adresse der Page Table in **Seitentabellenregister** laden
- Seitennummer = Index in Page Table $\rightarrow$ ein Eintrag pro Seite:
  - **P-Bit** (present) = Seite eingelagert?
  - Kachelnummer k
  - **R-Bit** (referenced) = Zugriff auf Seite?
  - **M-Bit** (modify) = Schreibzugriff? (bei Auslagerung zurückschreiben)

Paging

Page Table

- Verwaltung der Zuordnung von Seiten zu Kacheln
- Das BS verwaltet eine Page Table pro Prozess
- Bei einem Prozesswechsel wird die Anfangsadresse der neuen Prozess-Page-Table in das Seitentabellenregister der CPU geladen

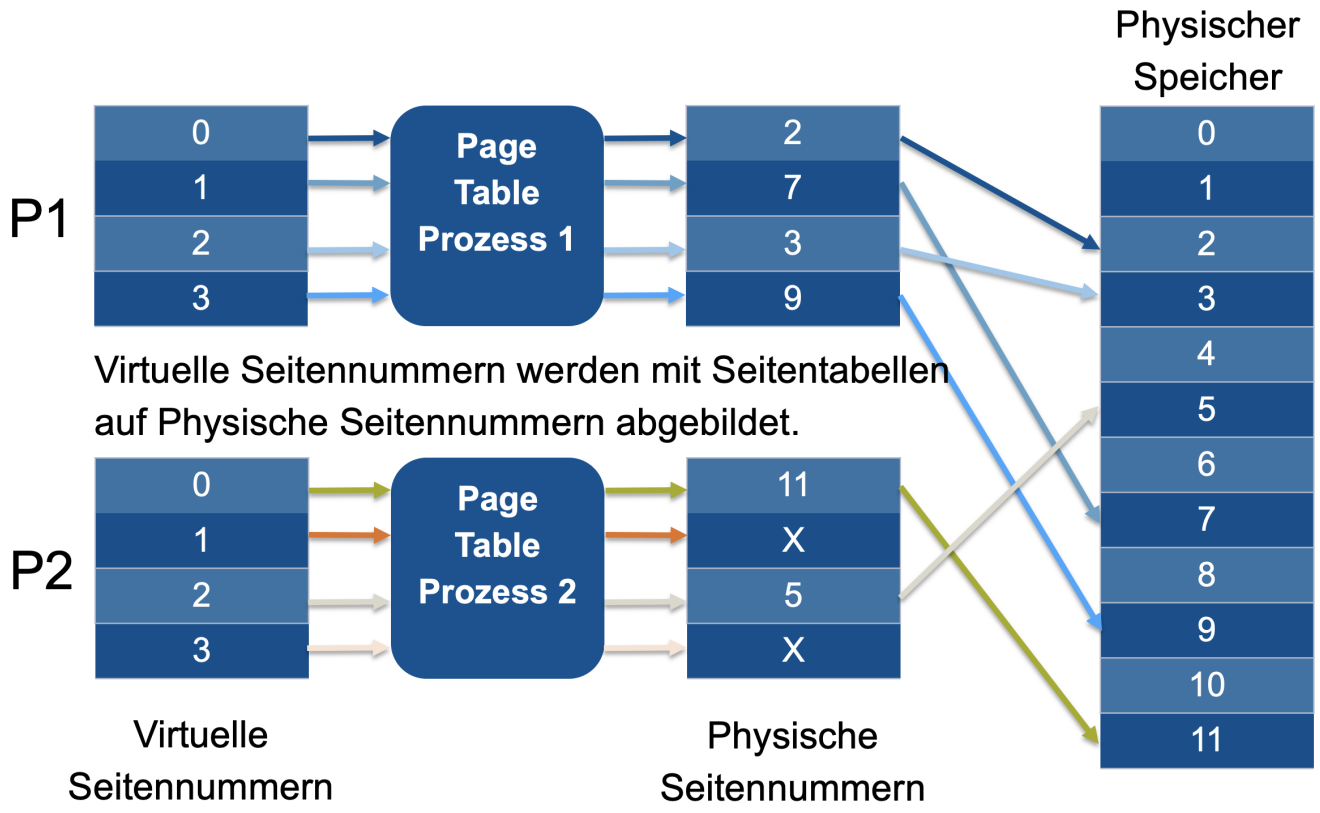
Seitenfehler (page fault)

- Bei einem Zugriff auf eine virtuelle Adresse einer Seite, die in keiner Kachel eingelagert ist
- Hardware löste einen Interrupt, einen page fault aus
- Das BS muss die erforderliche Seite einlagern

Seiteneinlagerung

- Falls alle Kacheln belegt sind, müssen Seiten ersetzt werden
- Auslagern von Seiten auf der Festplatte

Paging

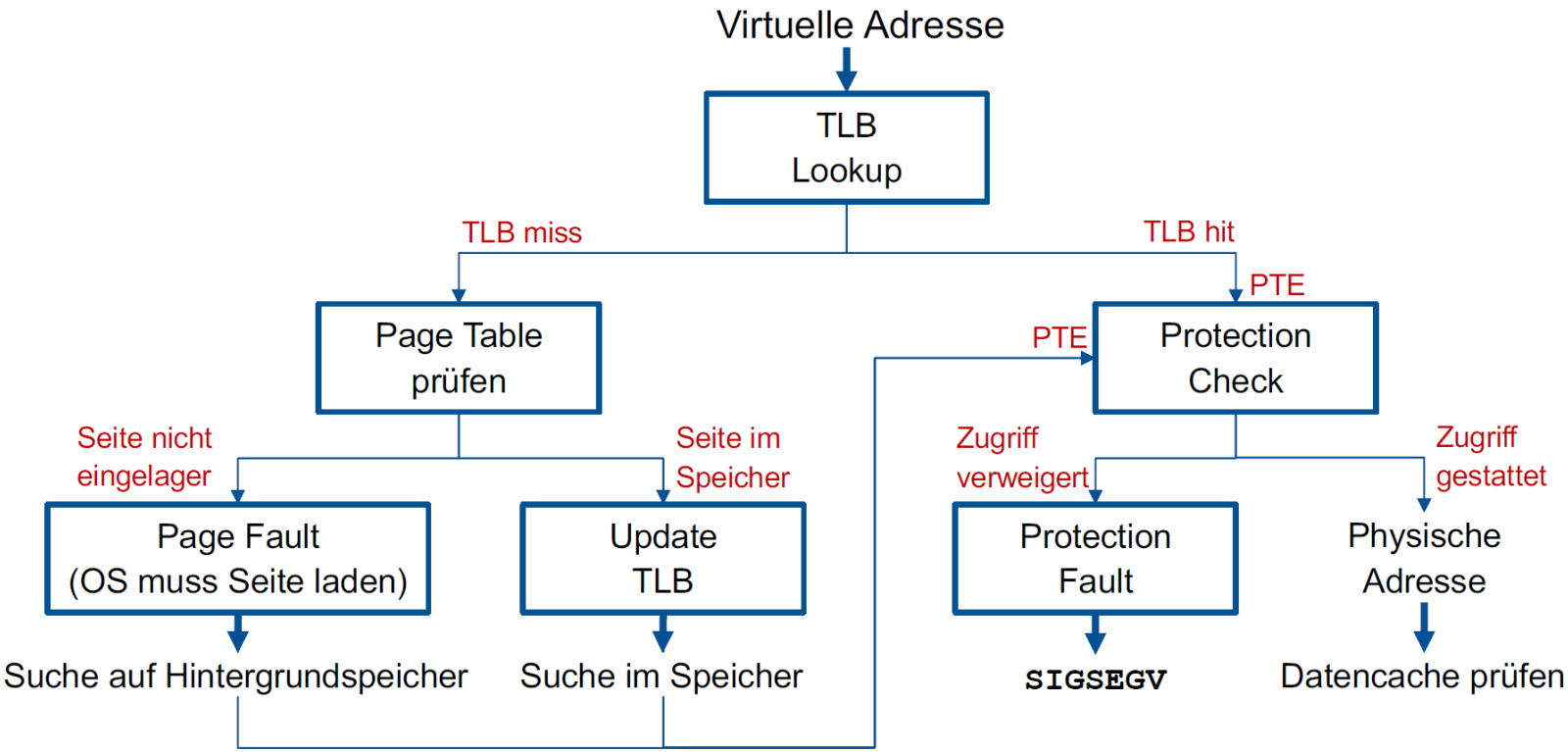

Physischer Speicher
P1
0
1
2
3
Page Table Prozess 1
2
7
3
9
Virtuelle Seitennummern werden mit Seitentabellen auf Physische Seitennummern abgebildet.
P2
0
1
2
3
Page Table Prozess 2
11
X
5
X
0
1
2
3
4
5
6
7
8
9
10
11
Virtuelle Seitennummern
Physische Seitennummern

Paging - MMU und TLB

Memory Management Unit (MMU): Hardware zur Seiten- $\rightarrow$ Kachelnummer Übersetzung

- Page Table im Hauptspeicher $\Rightarrow$ jeder Zugriff = 2 Speicherzugriffe (teuer!)
- **Translation Lookaside Buffer (TLB)** = Cache für Übersetzungen
- TLB-Hit: direkte Übersetzung, kein PT-Lookup
- TLB-Miss: Lookup in Page Table, Eintrag in TLB cachen

Paging - Adressübersetzung



Beispiel: Adressübersetzung

Gegeben: 16 KiB Hauptspeicher, 64 KiB Virtueller Speicher, 128 Seiten, 32 Kacheln

Schritt 1: Bitbreiten berechnen

- Seitengröße = Kachelgröße = $\frac{16\text{ KiB}}{32} = 512\text{ B} = 2^9 \Rightarrow$ **9 Bit Offset**
- 128 Seiten $= 2^7 \Rightarrow$ **7 Bit Seitennummer**
- 32 Kacheln $= 2^5 \Rightarrow$ **5 Bit Kachelnummer**

$\Rightarrow$ Virtuelle Adresse: 7 + 9 = **16 Bit**, Physische Adresse: 5 + 9 = **14 Bit**

Beispiel: Adressübersetzung

Schritt 2: Übersetze $v = \texttt{0x36B}$ mit folgender Page Table:

Seite	Kachel
1	10101
2	00111
3	01000
4	10001

Beispiel: Adressübersetzung

Schritt 2: Übersetze $v = \texttt{0x36B}$ mit folgender Page Table:

Seite	Kachel
1	10101
2	00111
3	01000
4	10001

a) Adresse in Binär (16 Bit):

$v = \texttt{0x36B} = \texttt{0000 0011 0110 1011}$

b) Aufteilen in Seitennr. (7 Bit) und Offset (9 Bit):

$v = \texttt{0000 001 1 0110 1011}$

Seite = 1 Offset = 0x16B

c) Kachelnummer aus Page Table:

Seite 1 $\rightarrow$ Kachel $\texttt{10101}_2 = \texttt{0x15}$

d) Physische Adresse zusammensetzen:

$p = \texttt{10101 1 0110 1011} = \underline{\texttt{0x2B6B}}$

Warum mehrstufiges Paging?

Problem: Einstufige Page Table für 64-Bit wäre riesig!

- 48-Bit virtueller Adressraum, 4 KiB Seiten $\Rightarrow 2^{36}$ Einträge
- Bei 8 Byte pro Eintrag = **512 GiB** pro Prozess nur für die Page Table!

Lösung: Mehrstufige Page Tables

- Hierarchische Struktur: nur benötigte Teile werden alloziert
- Jede Stufe indiziert die nächste Page Table
- Typisch: 4 Stufen bei 64-Bit Systemen

Aufbau einer 64-Bit virtuellen Adresse (vierstufig)

Beispiel: 12 Bit Offset, je 8 Bit pro Stufe (L1–L4), Rest ignoriert

ignored	L1 (8 Bit)	L2 (8 Bit)	L3 (8 Bit)	L4 (8 Bit)	Offset (12 Bit)
63–44	43–36	35–28	27–20	19–12	11–0

- Seitengröße = 2^{12} = **4 KiB**
- Einträge pro Tabelle = 2^{8} = **256**
- Physische Adresslänge: z.B. 40 Bit $\Rightarrow$ 28 Bit Kachelnummer + 12 Bit Offset

Mehrstufiges Paging - Übersetzungsablauf

Schritt für Schritt:

1. Virtuelle Adresse in Indizes aufteilen (L1, L2, L3, L4, Offset)
2. L1-Index $\rightarrow$ Eintrag in PT1 liefert **Adresse von PT2**
3. L2-Index $\rightarrow$ Eintrag in PT2 liefert **Adresse von PT3**
4. L3-Index $\rightarrow$ Eintrag in PT3 liefert **Adresse von PT4**
5. L4-Index $\rightarrow$ Eintrag in PT4 liefert **Frame-Nummer**
6. Physische Adresse = Frame-Nummer $\|$ Offset

Wichtig: Bei jedem Schritt kann ein **Page Fault** auftreten, falls der Eintrag ungültig ist (P-Bit = 0).

Mehrstufiges Paging - Beispiel (Retake 2024)

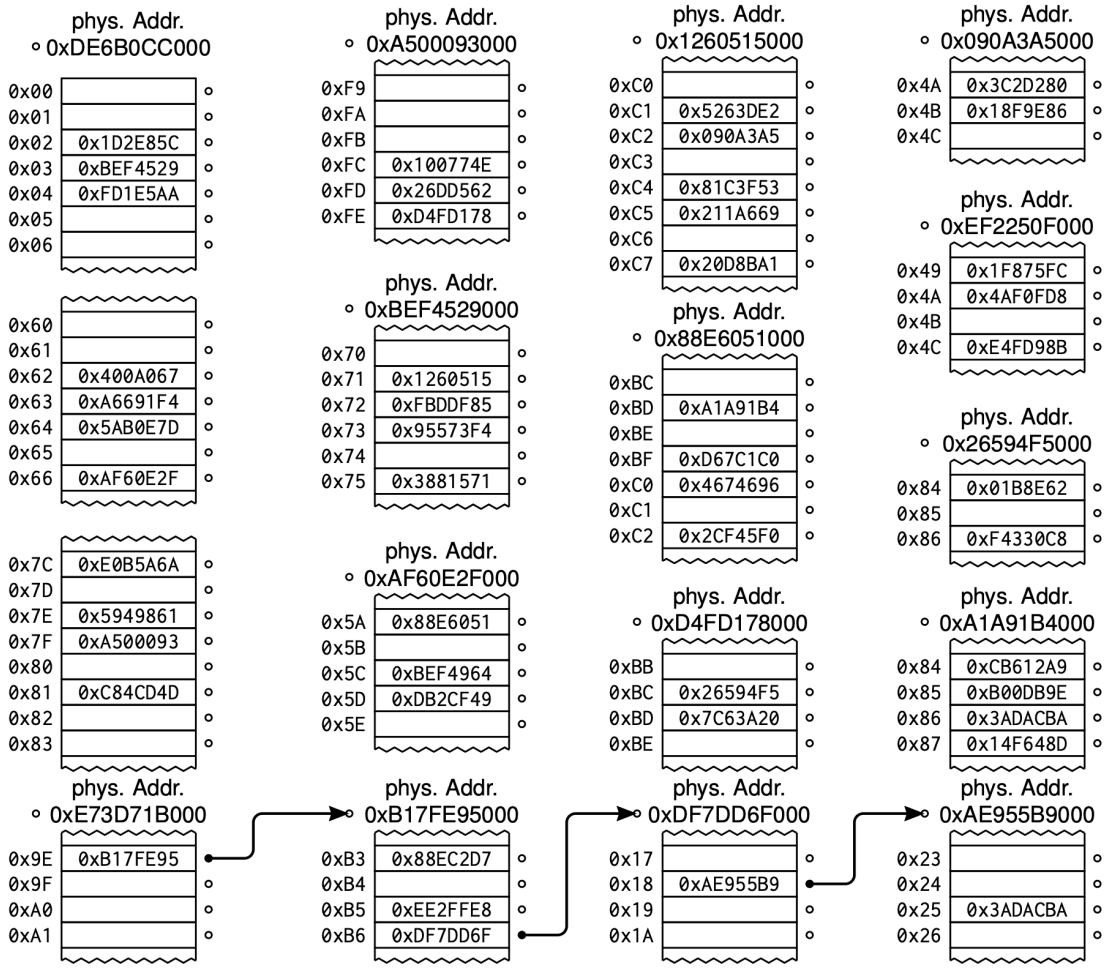


Abbildung 7.1: Seitentabellen von Prozess P_1 und Prozess P_2

Retake 2024, Aufgabe 7 - Speicherverwaltung (12 Punkte)

Gegeben sei ein 64-bit System, welches mittels eines **vierstufigen Paging-Verfahrens** virtuelle Adressen auf physische Adressen abbildet.

- Für den Offset entfallen **12 Bit**
- Für jede Stufe der Abbildung **8 Bit**
- Die verbleibenden Bits werden ignoriert
- Die physische Adresslänge beträgt **40 Bit**

a) Wie groß ist eine Seite? Verwenden Sie korrekte Binärpräfixe.

b) Wie viele Einträge hat eine Seitentabelle?

Retake 2024, Aufgabe 7 - Speicherverwaltung (12 Punkte)

Gegeben sei ein 64-bit System, welches mittels eines **vierstufigen Paging-Verfahrens** virtuelle Adressen auf physische Adressen abbildet.

- Für den Offset entfallen **12 Bit**
- Für jede Stufe der Abbildung **8 Bit**
- Die verbleibenden Bits werden ignoriert
- Die physische Adresslänge beträgt **40 Bit**

a) Wie groß ist eine Seite? Verwenden Sie korrekte Binärpräfixe.

2^{12} Bytes = **4 KiB**

b) Wie viele Einträge hat eine Seitentabelle?

Retake 2024, Aufgabe 7 - Speicherverwaltung (12 Punkte)

Gegeben sei ein 64-bit System, welches mittels eines **vierstufigen Paging-Verfahrens** virtuelle Adressen auf physische Adressen abbildet.

- Für den Offset entfallen **12 Bit**
- Für jede Stufe der Abbildung **8 Bit**
- Die verbleibenden Bits werden ignoriert
- Die physische Adresslänge beträgt **40 Bit**

a) Wie groß ist eine Seite? Verwenden Sie korrekte Binärpräfixe.

2^{12} Bytes = **4 KiB**

b) Wie viele Einträge hat eine Seitentabelle?

2^{8} = **256** Einträge

Retake 2024, Aufgabe 7 - Adressübersetzung

c) Übersetzen Sie die folgenden virtuellen Adressen in physische Adressen unter Zuhilfenahme der Seitentabellen. **Hinweise:**

- Die Seitentabelle der ersten Stufe befindet sich an der physischen Adresse `0xDE6B0CC000`
- Falls ein Pagefault auftritt, schreiben Sie “Pagefault” anstelle der physischen Adresse
- Notieren Sie Ihre Lösung mitsamt Zwischenergebnissen

Virt.Addr.	Index Stufe 1	Index Stufe 2	Index Stufe 3	Index Stufe 4	phys. Addr.
`0x7FFEBC84420`					
`0x0371C24AF59`					

Seitentabellen (Referenz)

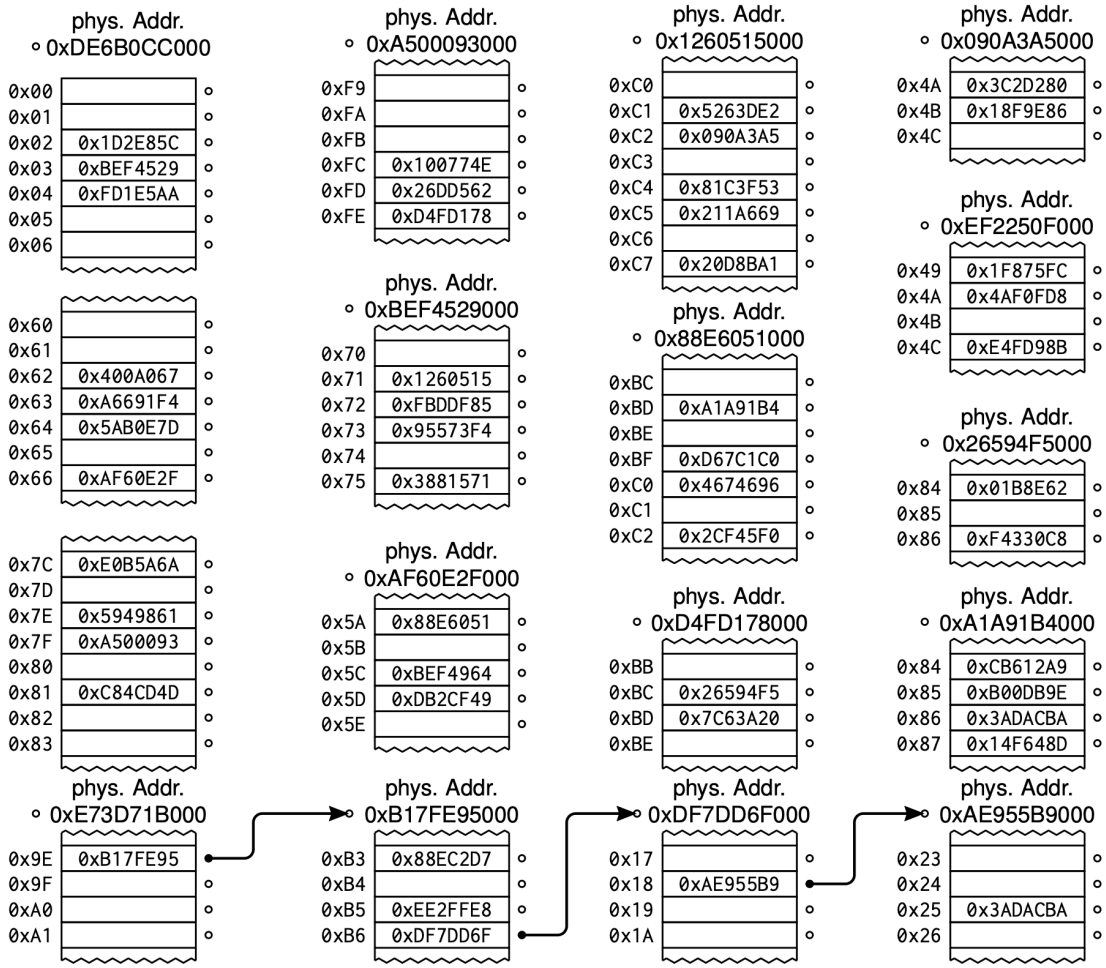


Abbildung 7.1: Seitentabellen von Prozess P_1 und Prozess P_2

Retake 2024, Aufgabe 7 - Adressübersetzung

c) Übersetzen Sie die folgenden virtuellen Adressen in physische Adressen unter Zuhilfenahme der Seitentabellen.

Virt.Addr.	Index Stufe 1	Index Stufe 2	Index Stufe 3	Index Stufe 4	phys. Addr.
`0x7FFEBC84420`	0x7F				
`0x0371C24AF59`	0x03				

Seitentabellen (Referenz)

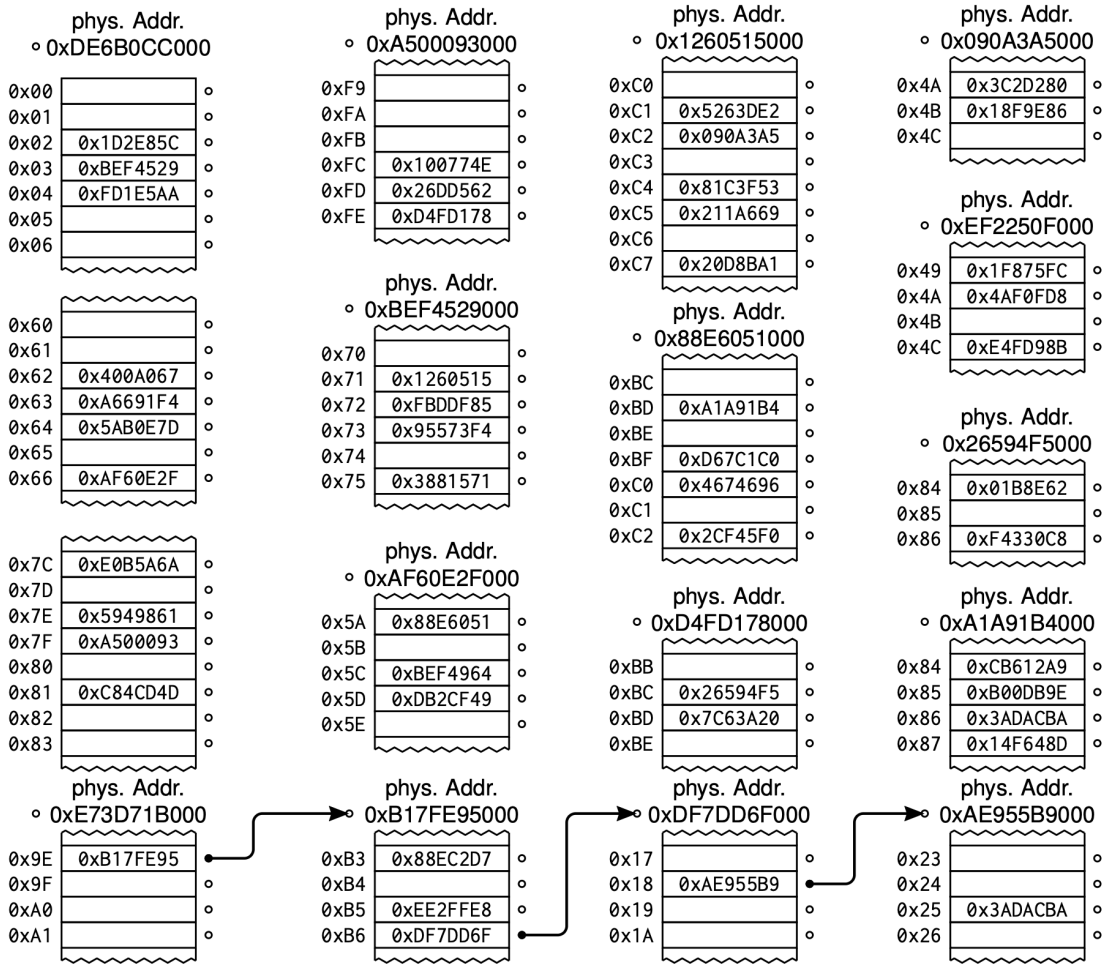


Abbildung 7.1: Seitentabellen von Prozess P_1 und Prozess P_2

Retake 2024, Aufgabe 7 - Adressübersetzung

c) Übersetzen Sie die folgenden virtuellen Adressen in physische Adressen unter Zuhilfenahme der Seitentabellen.

Virt.Addr.	Index Stufe 1	Index Stufe 2	Index Stufe 3	Index Stufe 4	phys. Addr.
`0x7FFEBC84420`	0x7F	0xFE			
`0x0371C24AF59`	0x03	0x71			

Seitentabellen (Referenz)

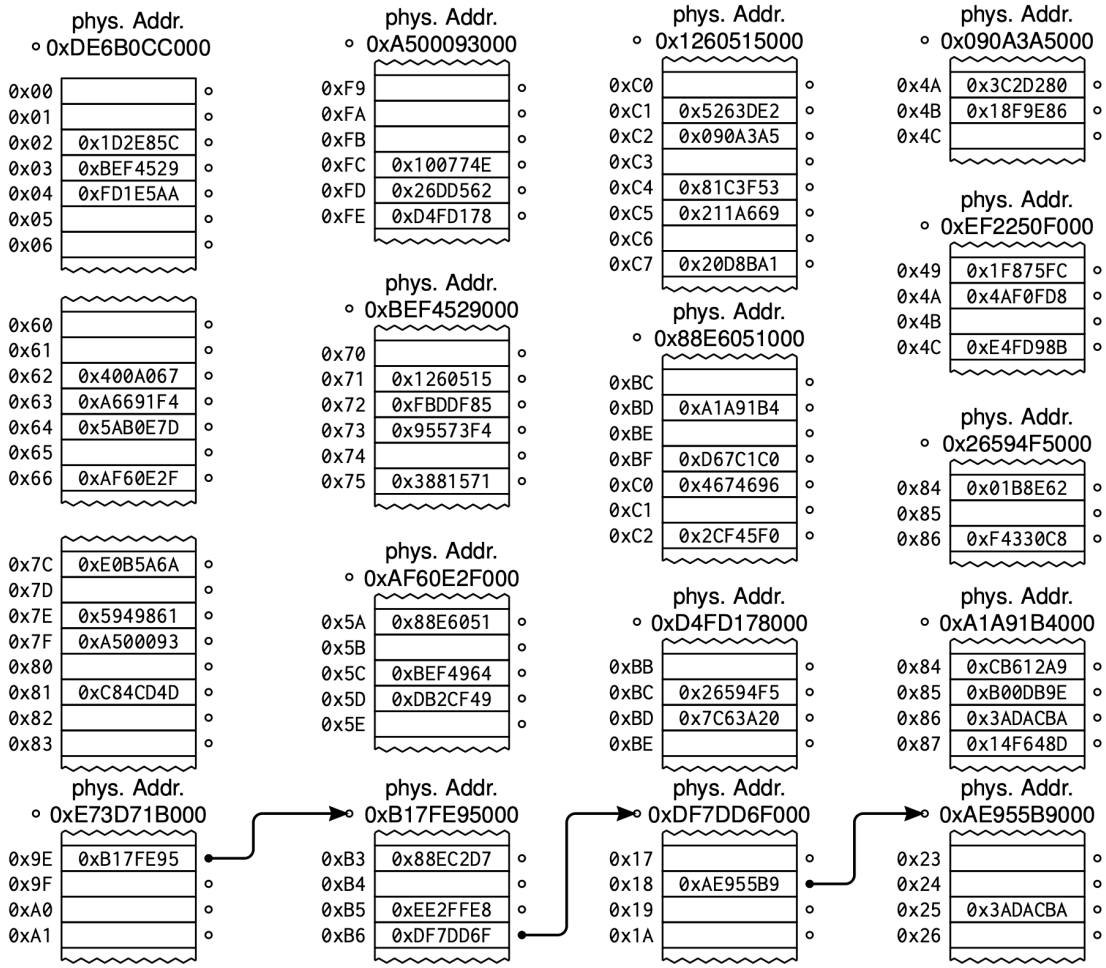


Abbildung 7.1: Seitentabellen von Prozess P_1 und Prozess P_2

Retake 2024, Aufgabe 7 - Adressübersetzung

c) Übersetzen Sie die folgenden virtuellen Adressen in physische Adressen unter Zuhilfenahme der Seitentabellen.

Virt.Addr.	Index Stufe 1	Index Stufe 2	Index Stufe 3	Index Stufe 4	phys. Addr.
`0x7FFEBC84420`	0x7F	0xFE	0xBC		
`0x0371C24AF59`	0x03	0x71	0xC2		

Seitentabellen (Referenz)

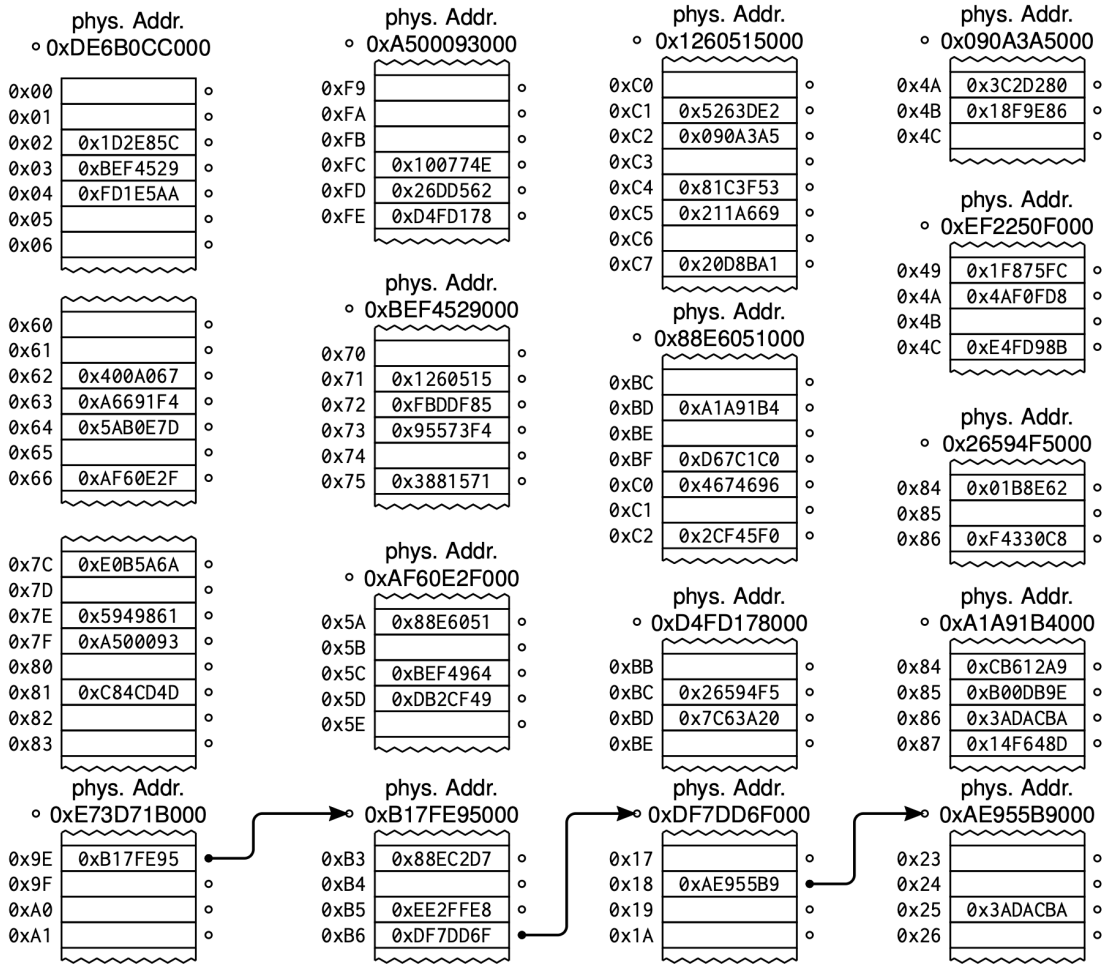


Abbildung 7.1: Seitentabellen von Prozess P_1 und Prozess P_2

Retake 2024, Aufgabe 7 - Adressübersetzung

c) Übersetzen Sie die folgenden virtuellen Adressen in physische Adressen unter Zuhilfenahme der Seitentabellen.

Virt.Addr.	Index Stufe 1	Index Stufe 2	Index Stufe 3	Index Stufe 4	phys. Addr.
`0x7FFEBC84420`	0x7F	0xFE	0xBC	0x84	
`0x0371C24AF59`	0x03	0x71	0xC2	0x4A	

Seitentabellen (Referenz)

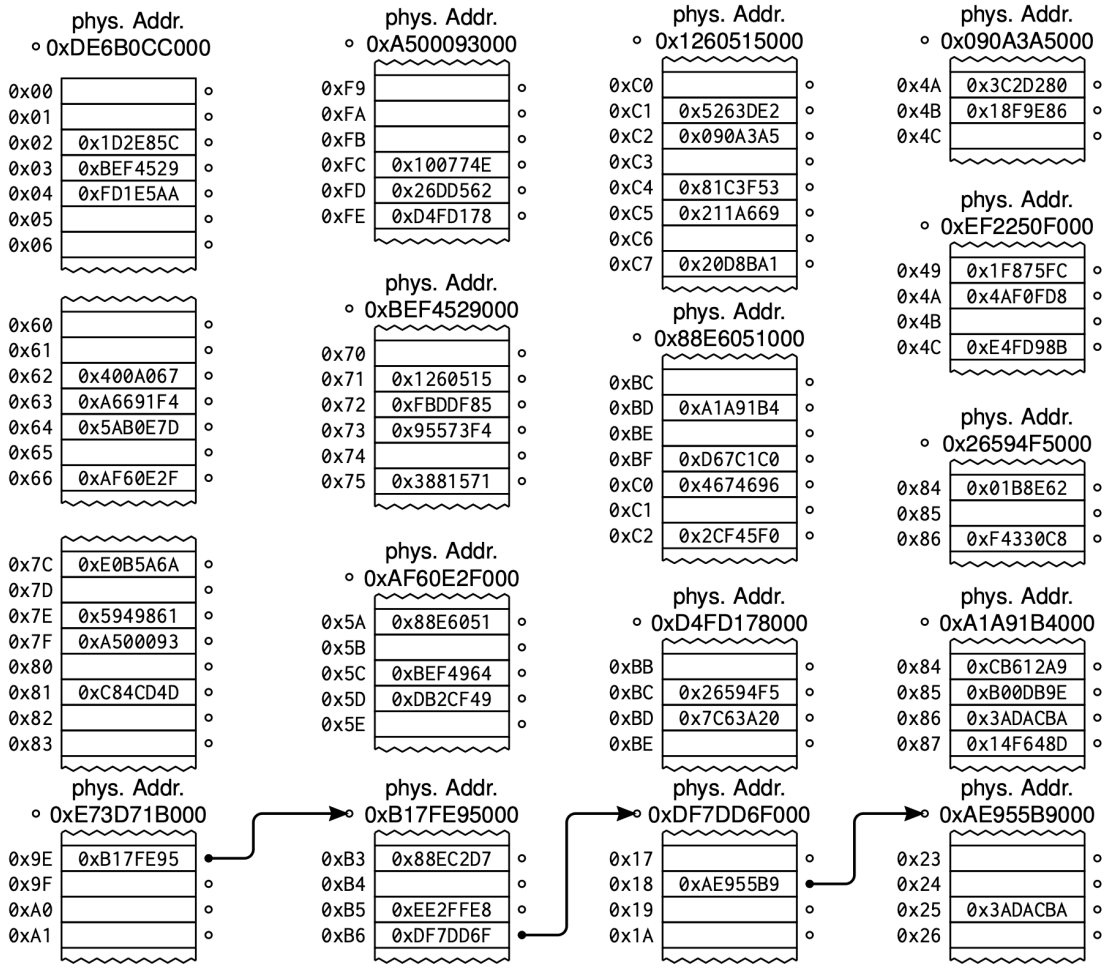


Abbildung 7.1: Seitentabellen von Prozess P_1 und Prozess P_2

Retake 2024, Aufgabe 7 - Adressübersetzung

c) Übersetzen Sie die folgenden virtuellen Adressen in physische Adressen unter Zuhilfenahme der Seitentabellen.

Virt.Addr.	Index Stufe 1	Index Stufe 2	Index Stufe 3	Index Stufe 4	phys. Addr.
`0x7FFEBC84420`	0x7F	0xFE	0xBC	0x84	`0x01B8E62420`
`0x0371C24AF59`	0x03	0x71	0xC2	0x4A	`0x3C2D280F59`

Seitentabellen (Referenz)

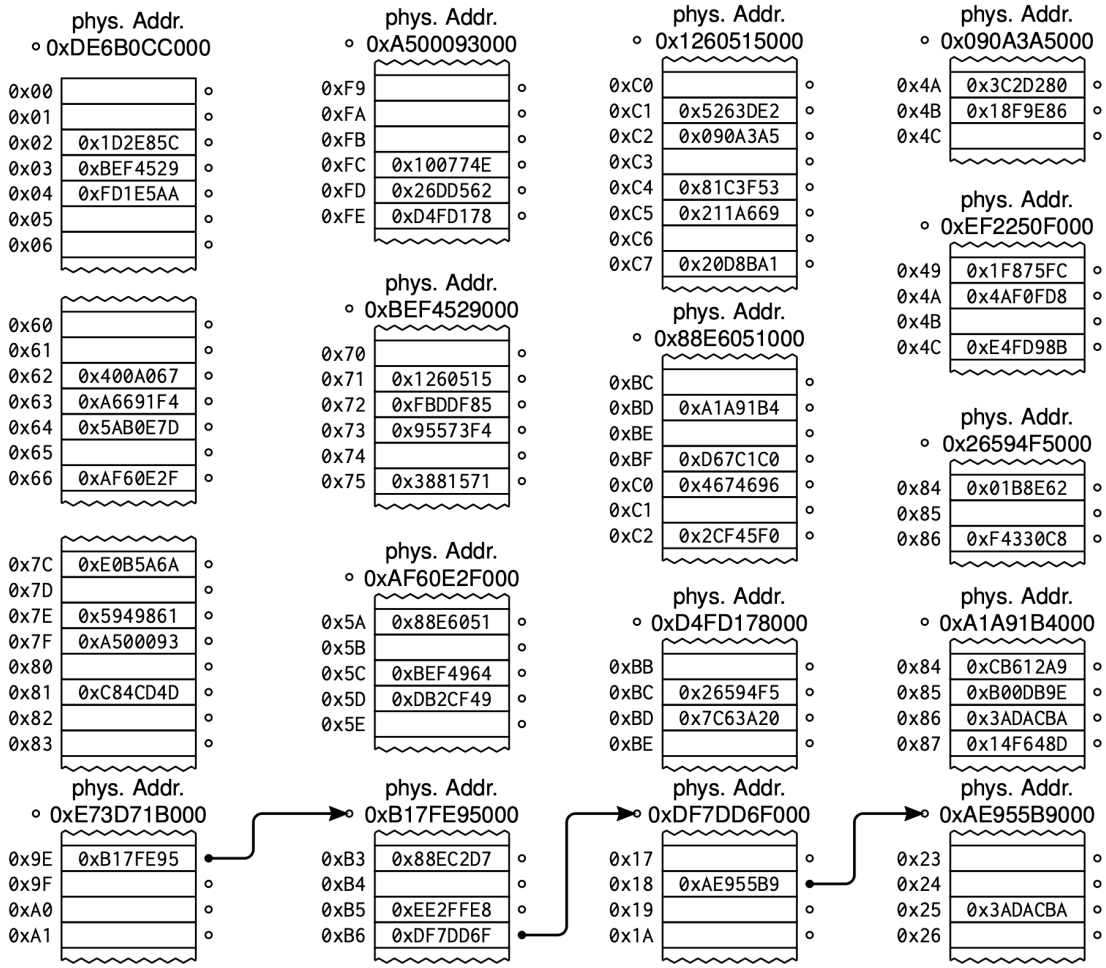


Abbildung 7.1: Seitentabellen von Prozess P_1 und Prozess P_2

Retake 2024, Aufgabe 7 - Lösungsweg

Aufbau einer 64-Bit virtuellen Adresse (vierstufig):

ignored	L1 (8 Bit)	L2 (8 Bit)	L3 (8 Bit)	L4 (8 Bit)	Offset (12 Bit)
63-44	43-36	35-28	27-20	19-12	11-0

Beispiel: v_1 = `0x7FFEBC84420`

Binär: `0111 1111 1111 1110 1011 1100 1000 0100 0100 0010 0000`

- L1-Index: `0x7F` $\rightarrow$ Eintrag in PT1 liefert Adresse von PT2
- L2-Index: `0xFE` $\rightarrow$ Eintrag in PT2 liefert Adresse von PT3
- L3-Index: `0xBC` $\rightarrow$ Eintrag in PT3 liefert Adresse von PT4
- L4-Index: `0x84` $\rightarrow$ Eintrag in PT4 liefert Frame-Nummer
- Offset: `0x420` $\rightarrow$ wird übernommen

$\Rightarrow$ phys. Adresse: Frame-Nummer || Offset = `0x01B8E62420`

Seitenersetzungsstrategien

Vorbemerkung

- Optimale Strategie: Auslagern derjenigen Seite, deren nächster Zugriff am weitesten in der Zukunft liegt
- Problem: Nicht implementierbar, BS kann keine Voraussagen machen über künftige Seitenzugriffe
- Algorithmus, der die optimale Strategie approximiert

Ersetzungsstrategien

- FIFO: First-In, First-Out
- Second-Chance
- Clock
- Least Recently Used, Aging
- Working Set (mit oder ohne Clock)

Second-Chance-Algorithmus

- FIFO lagert die älteste Seite aus: Modifikation
- Wenn älteste Seite das R-Bit gesetzt hat: Seite erhält eine *Second Chance*
  - lösche dies und verschiebe die Seite an das Ende der Liste
  - Aktualisiere Einlagerungszeit
  - Mit der Suche fortfahren
- Andernfalls:
  - Seiteninhalt zurückschreiben, wenn M-Bit gesetzt
  - Sonst Seiteninhalt verwerfen
- Anmerkung
  - Wenn alle R-Bits gesetzt: Second Chance = FIFO
  - Problem: Seiten in der Liste werden unnötig oft verschoben
  - Lösung: Effiziente Alternative zur Liste: Clock-Algorithmus

Clock-Algorithmus

Konzept

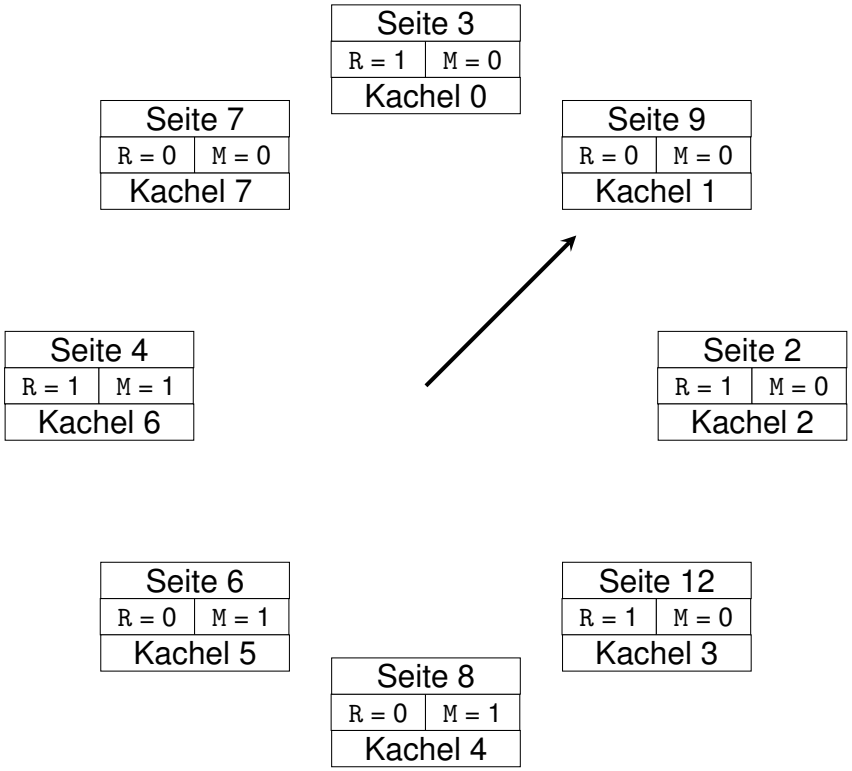
- Verwalten der Seiten in einer zirkulären Liste, angeordnet wie auf einem Ziffernblatt
- Nutzen eines (Uhr-)Zeigers, der auf die älteste Seite verweist

Bei Page Fault:

1. Falls R-Bit nicht gesetzt: Ersetze Seite durch neue Seite und rücke den Zeiger um eine Position weiter
   1.1 Falls M-Bit gesetzt: Inhalte der Seite zurück in den Hintergrundspeicher schreiben
   1.2 Sonst: verwerfe Seiteninhalte
2. R-Bit auf 0 setzen und Zeiger eine Position weiter
3. Zurück zu Schritt 1

Aufgabe Clock-Algorithmus (Endterm 2024):

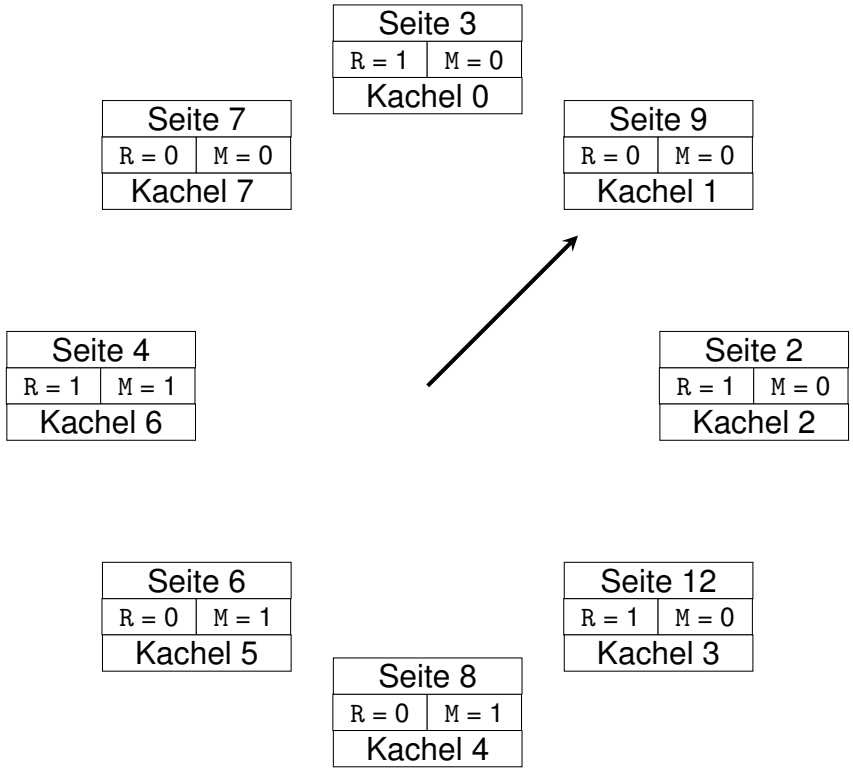
Die Uhr tickt



Hinweis: R=Referenced, M=Modified. Der Zeiger muss **immer** angegeben werden.

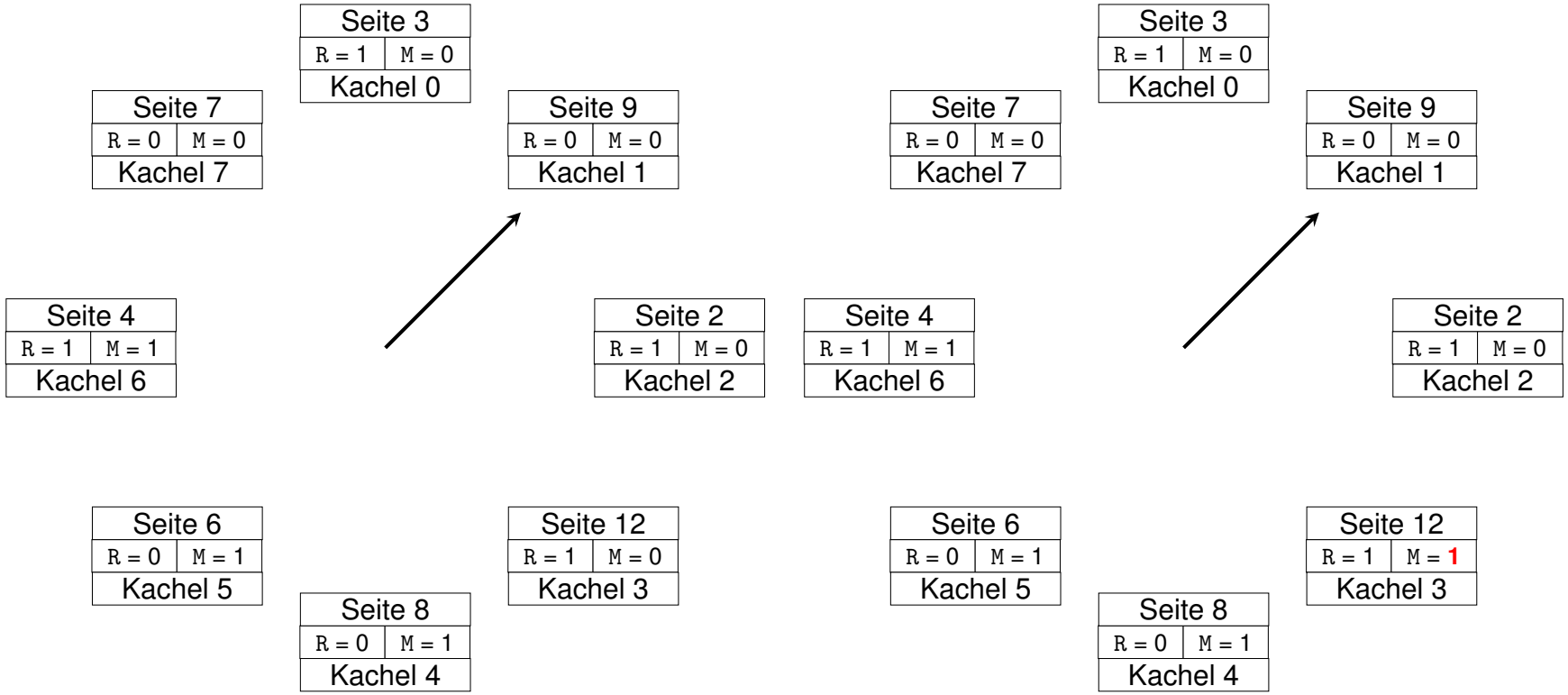
Aufgabe Clock-Algorithmus:

a)* Schreibzugriff auf Seite 12.



Aufgabe Clock-Algorithmus:

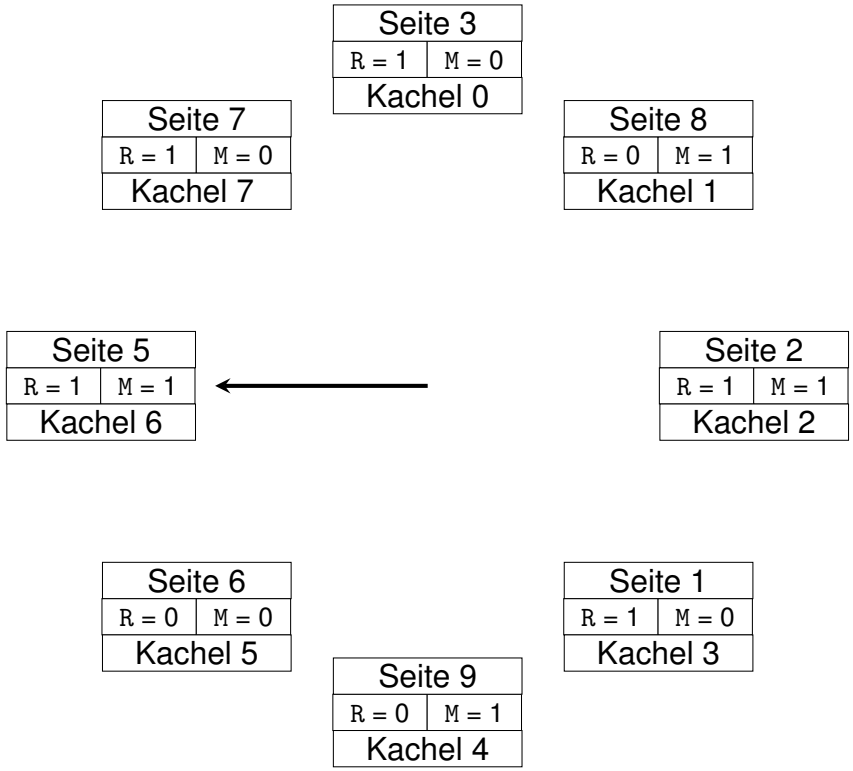
a)* Schreibzugriff auf Seite 12.



Seite 12 ist bereits geladen $\Rightarrow$ nur M-Bit setzen, Zeiger bleibt.

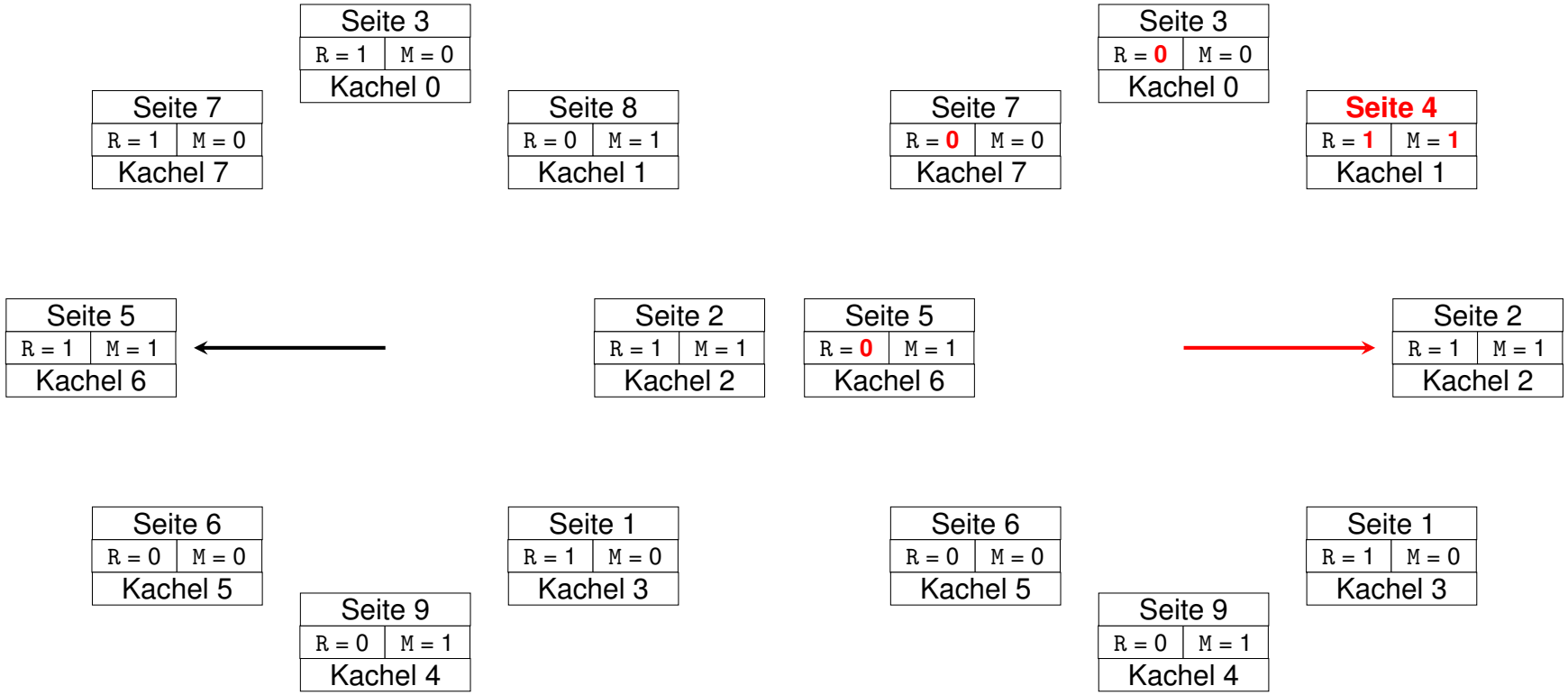
Aufgabe Clock-Algorithmus:

b)* Schreibzugriff auf Seite 4.



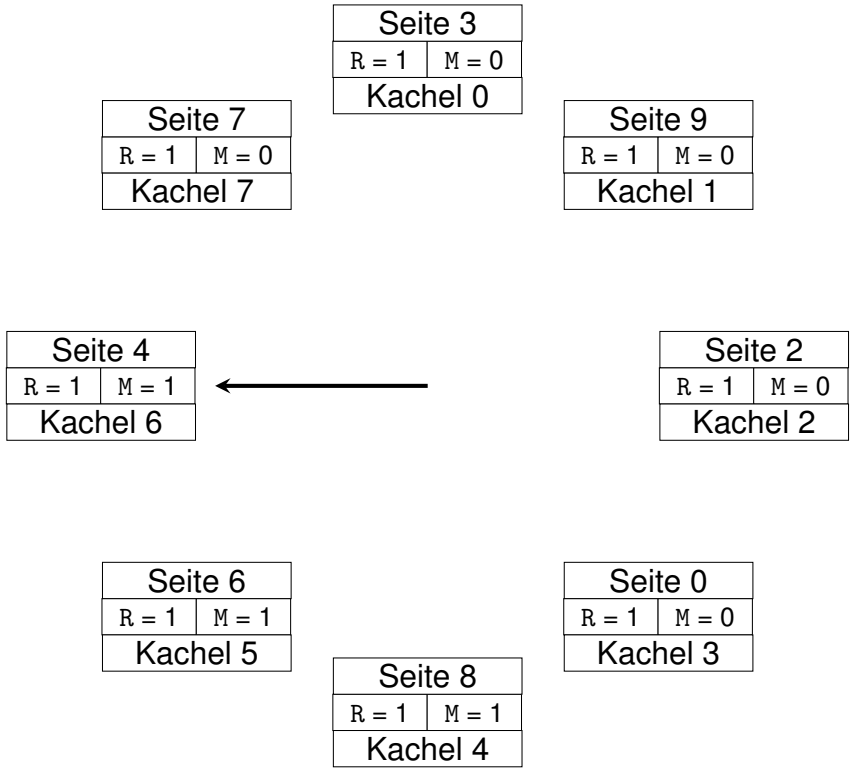
Aufgabe Clock-Algorithmus:

b)* Schreibzugriff auf Seite 4.



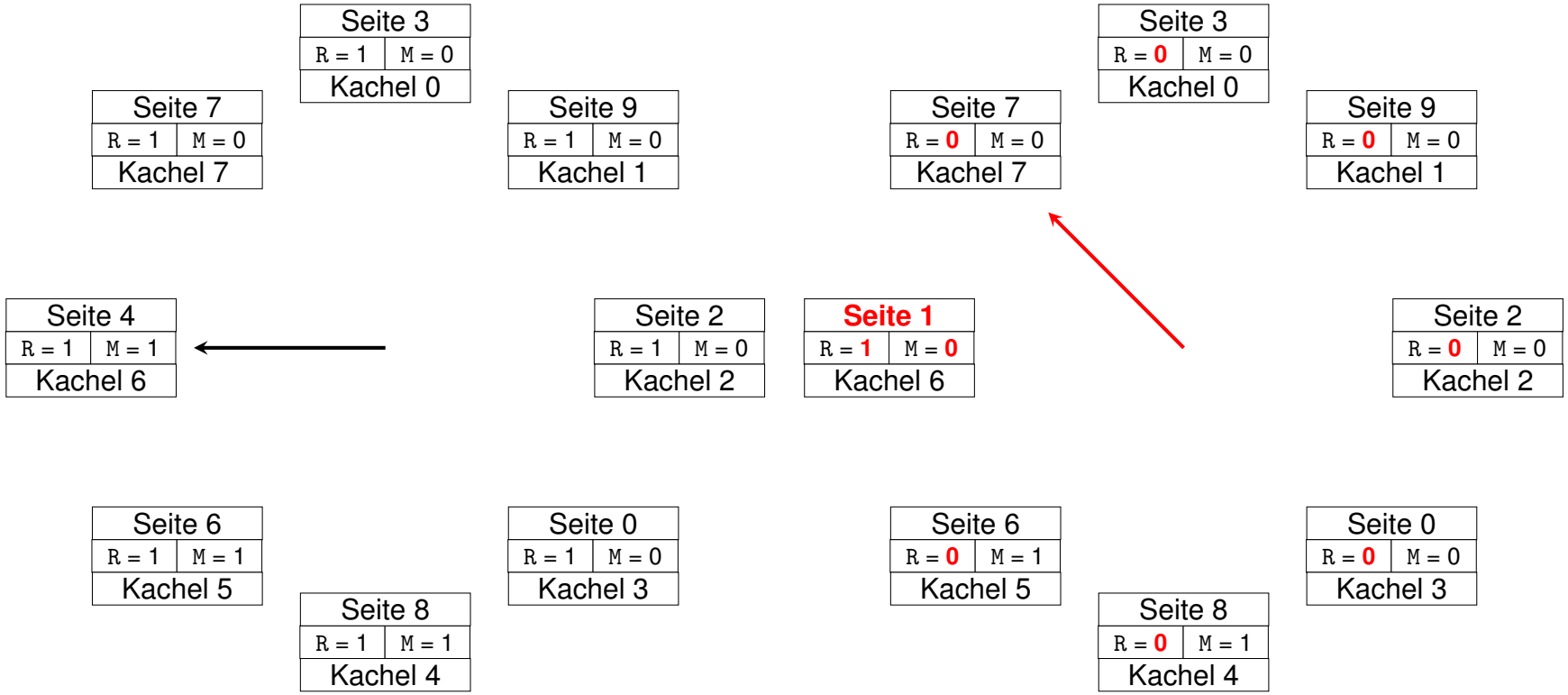
Aufgabe Clock-Algorithmus:

c)* Lesezugriff auf Seite 1.



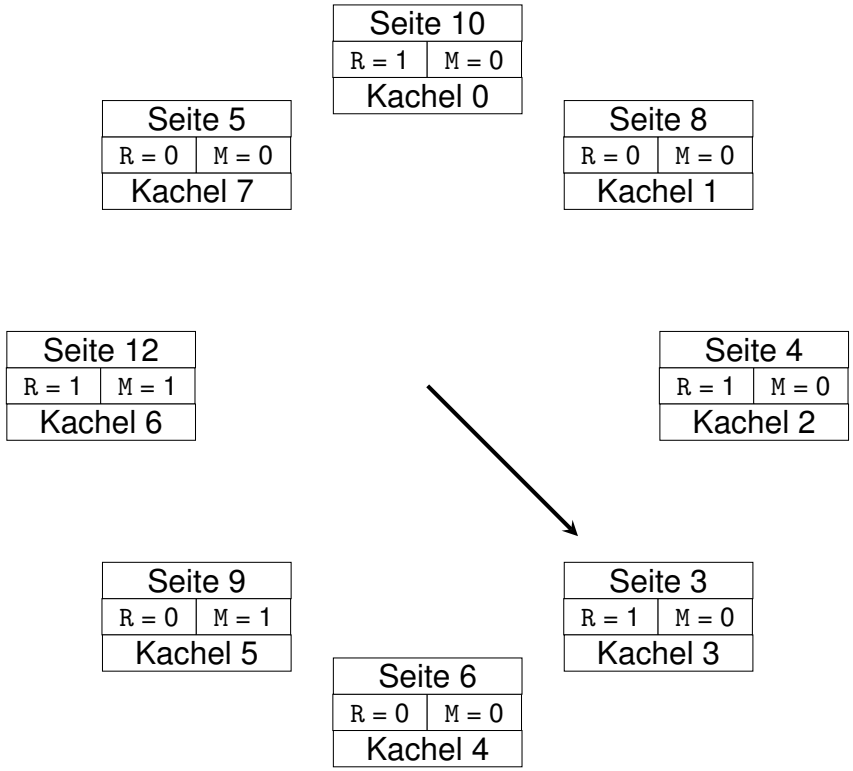
Aufgabe Clock-Algorithmus:

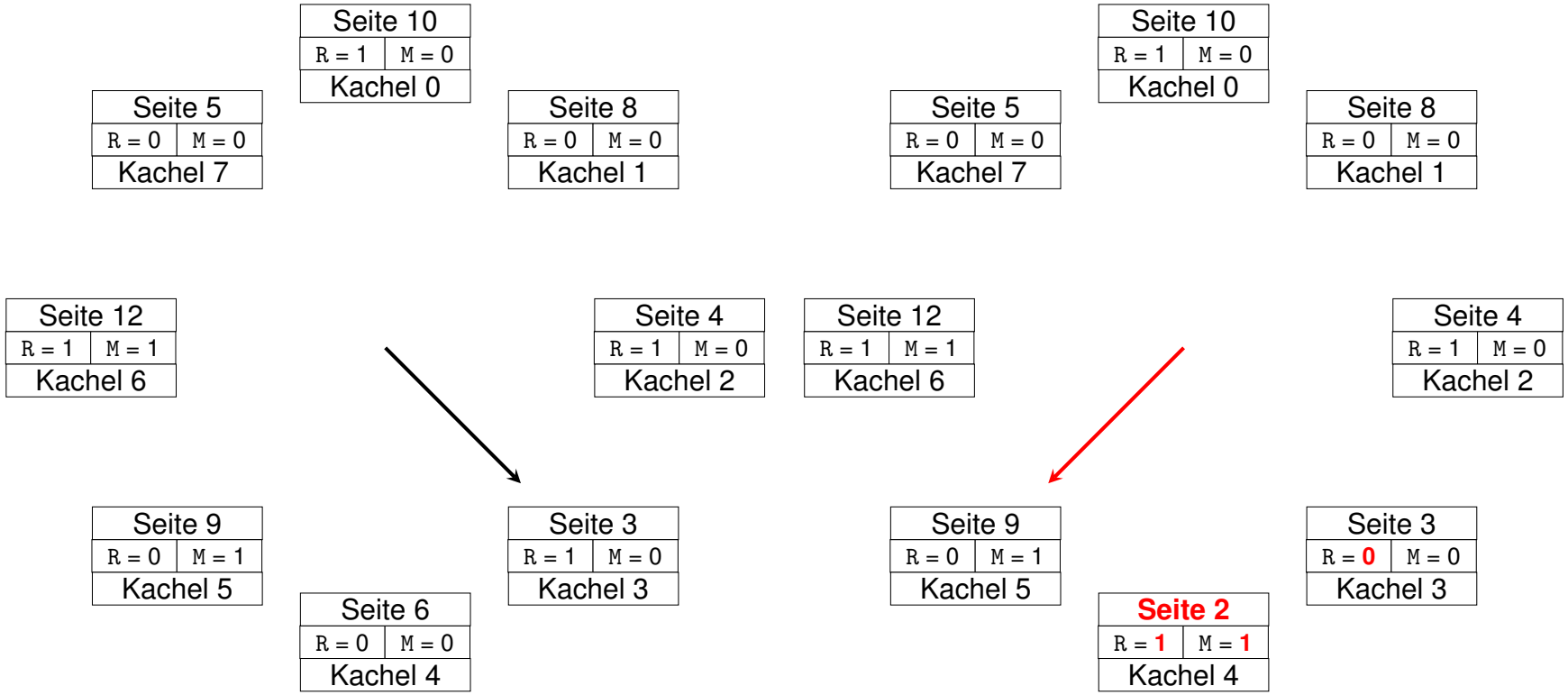
c)* Lesezugriff auf Seite 1.



Aufgabe Clock-Algorithmus:

d)* Schreibzugriff auf Seite 2.



Aufgabe Clock-Algorithmus:

d)* Schreibzugriff auf Seite 2.